

AMS

JavaScript: The Language Behind the Platform

A teaching guide for managers and new engineers, written entirely from the live AMS codebase.

Airline Management System
Finance · Fleet · Procurement
JavaScript (ESM) · Express · Knex · PostgreSQL 16 · React

Every code sample in this document was copied from the running system. Nothing here is invented illustration.

How to use this guide

This document teaches JavaScript using the AMS airline platform as the worked example. Every concept is introduced in plain language, demonstrated in real AMS code, and then restated in terms of what it means for the business.

It assumes you have never written a program. It does not assume you are technical. If you can read a sentence and follow an argument, you can read this.

Three ways to read it

If you are...	Start at	You will get
A manager, no coding background	Part I, then Parts III and V, then Part XIII	Enough to review AMS output critically and ask the right questions of your engineers
A new engineer joining the team	The whole document, in order	A working grasp of the language and the codebase conventions
A developer reviewing AMS code	Parts XI, XII, XIII	The reasoning behind the testing and typing decisions, and the defects to look for

What is in each part

Part	Subject	Why it matters
I	Orientation	What the system is for and why it is built this way
II	The JavaScript language	The fundamentals, using AMS code throughout
III	Money	The single most important discipline in this codebase
IV	Airline metrics	The ratios every commercial decision rests on
V	The costing engine	How AMS answers "what did that flight cost?"
VI	Fuel	The largest and least controllable cost
VII	Airworthiness	Where a software error can cost an operator its licence
VIII	Algorithms	Route planning and fuel optimisation, and how they were proven
IX	The server and the database	How AMS stops an airline overspending
X	The audit trail	How history is made unchangeable
XI	Types without TypeScript	How the team gets safety without a build step
XII	Testing	The actual safety net behind every number
XIII	Defects	Real bugs that shipped, and what each one teaches
XIV	What comes next	Planned work, and what still needs qualified people

A note on what this document is not

AMS makes calculations that support financial and regulatory decisions. This guide explains how the code works. It is not accounting advice, airworthiness advice, or regulatory advice, and nothing in it substitutes for a qualified aviation finance or compliance professional signing off on the system before it influences a real decision.

Contents

Part I — How to read this guide

- Who this is for, and how to use it
- What AMS actually does
- Why this system is written in JavaScript
- Reading the code samples

Part II — The language, from the beginning

- What a variable actually is
- Numbers are the one thing to fear
- Text, and how code reads it
- Decisions: if, and the shape of a condition
- Loops, and when not to use one
- Functions: naming a piece of work
- Objects: labelled collections
- Arrays: ordered collections
- The special value undefined, null, and NaN
- Modules: how files talk to each other

Part III — Money, the discipline that defines this codebase

- Why nobody uses decimals for money in code
- Cents: the integer unit AMS chose
- Parsing a human amount safely
- Rounding: the rules AMS applies
- Splitting money without losing a cent
- Percentages as integers

Part IV — Measuring an airline

- The industry ratios, explained
- Asking for capacity: ASK and RPK
- Cost and revenue per unit: CASK and RASK
- Why you must never average an average
- Break-even load factor

Part V — The costing engine

- Two kinds of cost
- Building the cost of one sector
- Allocating fixed cost to flights
- Full profit and loss for a flight
- Marginal cost: what one more passenger costs
- Aggregating many flights
- The route verdict

Part VI — Fuel, the largest single cost

- Uplift is not burn
- Reconciling plan against actual
- Tankering: carrying fuel is a decision
- Statistical detection of a fuel problem

Part VII — Airworthiness and legal safety

- Life-limited parts and three counters
- Finding the limit that binds first
- Directives and bulletins are not the same
- Deferral deadlines and the off-by-one
- Why a deferral cannot cancel a directive
- The dispatch gate

Part VIII — Routing and fuel planning (the algorithms)

- The problem, stated precisely
- The naive approaches and why they fail
- Keeping only useful possibilities
- The dominance rule and why it is sound
- A second solver to check the first
- Brute force as an oracle
- Optimal uplift planning
- Proving the uplift policy is optimal
- The five real bugs these tests caught

Part IX — The server side: money in a database

- What a server is
- Asynchronous JavaScript and promises
- Database queries as chainable objects
- The race that overspends an airline
- Row locks and transactions
- Idempotency: making retries safe
- All-or-nothing across a flight leg

Part X — The audit trail

- Why an audit trail must be unchangeable
- Hashing: a fingerprint for data
- Chaining entries together
- Canonical form: making hashing reliable
- Verifying a chain

Part XI — Types without TypeScript

- The decision to write plain JavaScript
- What JSDoc does and does not do
- The type-check gate
- Where the guarantee really comes from

Part XII — Testing as the real safety net

- What a test suite is for
- Structural tests versus expected values
- Testing against three implementations
- Property tests: invariants that must hold
- Making random tests reproducible
- Regression tests for real defects

Part XIII — Defects found, and what they teach

Defect 1: an object built from nothing
Defect 2: an answer multiplied by zero
Defect 3: the test that shared the bug
Defect 4: the wrong node in the solver
Defect 5: refusing the plans it existed for
Defect 6: an infinite loop in a legal case
What the pattern tells you

Part XIV — What comes next

The server layer, when it is built
Identity, access and multi-tenancy
Spreadsheets, PDF and Word generation
What still needs a qualified human
Glossary

Part I — How to read this guide

What AMS actually does

An airline knows roughly what it spent last year. What it usually cannot tell you is what a particular flight cost, why a route is losing money, or whether a purchase was actually approved.

The reason is timing. A fuel invoice for January arrives in March, combined across hundreds of flights. By the time finance opens it, the aircraft that burned that fuel has flown several hundred more sectors. There is no honest way to attribute the cost to the flight that caused it, so nobody tries, and the number simply does not exist.

```
/**
 * AMS — Costing engine. THE SPINE OF THE PLATFORM.
 *
 * Every other module hangs off this one. Fuel, fleet, crew allocation,
 * route profitability, break-even decisions, executive reporting and
 * the budget all reconcile through the arithmetic in this file.
 *
 * The central idea: an airline cannot answer "what did this flight
 * cost?" by looking at invoices. Invoices arrive weeks later, at
 * monthly or quarterly grain, aggregated across hundreds of rotations.
 * By the time finance sees the Jet A invoice, the aircraft that burned
 *   it has flown 300 more sectors.
```

shared/src/costing.js — the opening of the module. Note that the first thing any engineer reads is a statement of the problem, not a list of functions.

AMS inverts this. Cost is attributed to a flight at the moment it happens, from primary records — the fuel uplift note, the crew duty report, the landing fee receipt. The accounting ledger then becomes a summary of facts that are already attributed, rather than a separate exercise in guessing.

This is worth pausing on, because it explains a great deal about how the code is written. When a fact is captured correctly at the moment it occurs, most later problems disappear. When it is not, they are almost impossible to fix retrospectively.

Why this system is written in JavaScript

The decision was deliberate and is recorded as ADR-001. The short version: AMS ships no build step. What you read in the repository is what runs.

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true,
    "noEmit": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": false,
    "skipLibCheck": true
  },
  "include": ["shared/src/**/*.js", "backend/src/**/*.js", "tests/**/*.js"],
  "exclude": ["node_modules"]
}
```

tsconfig.json — the whole type-safety configuration. There is no "outDir" and no compiler step, because nothing is compiled.

- **No build step.** No transpiler, no bundler, no generated files. A change is live when the file is saved.
- **No second language.** No TypeScript files alongside JavaScript files, so nothing drifts out of sync.
- **Types are documented and checked** using annotations that live inside the JavaScript itself (Part XI).
- **The check is automated.** It runs in CI and fails the build on a type error.

An honest weakness in that configuration

The strict option is currently set to false. That means null and undefined are not rigorously checked. For a codebase that handles money and airworthiness safety, strict: true would be the safer setting and would likely surface genuine defects. This is a known gap, recorded here rather than quietly omitted.

Reading the code samples

Samples are reproduced exactly as they appear in the repository, including comments. Where a sample is long, only the relevant part is shown.

```
export function cents(input) {
  const raw = typeof input === 'number' ? input.toFixed(2) : String(input);
  const cleaned = raw.replace(/[\s, _]/g, '');
  const match = /^(?)(\d*)(?:\.(?)(\d{1,2}))?$/ .exec(cleaned);
  if (!match) throw new MoneyError(`Cannot parse monetary amount: "${input}"`, 'MONEY_UNPARSEABLE');

  const [, sign, whole = '0', frac = ''] = match;
  if (whole === '' && frac === '') {
    throw new MoneyError(`Cannot parse monetary amount: "${input}"`, 'MONEY_UNPARSEABLE');
  }
  const frac2 = (frac + '00').slice(0, 2);
  const value = Number(whole || '0') * 100 + Number(frac2);
  const signed = sign === '-' ? -value : value;
  if (!Number.isSafeInteger(signed)) {
    throw new MoneyError(`Monetary amount out of safe integer range: "${input}"`, 'MONEY_OVERFLOW');
  }
  return signed;
}
```

shared/src/money.js — turns the text "1,234.56" into the integer 123456. Part III explains why this function exists at all.

A word on how to read the rest of this

Some parts of this guide describe genuinely hard mathematics — particularly Part VIII, on route planning. Those sections are written so that the *argument* can be followed without the algebra. The point is not for you to be able to re-derive the proof at your desk. The point is that you should be able to understand why the engineers trust it, and therefore whether you should.

Where something is a judgement call rather than a fact, this document says so. Part XIII documents six real defects found in this codebase, including two where the code and its tests were wrong in the same way. A system that has never been wrong has not been tested.

Part II — The language, from the beginning

JavaScript has about a dozen core ideas. This part covers all of them, using code from AMS. Each section introduces the idea in plain language first, then shows it in real code.

What a variable actually is

A variable is a name attached to a value. That is genuinely all it is. The name lets you refer to the value later without repeating it.

```
const PASSENGER_ALL_UP_KG = 100;    // ~75 kg person + ~25 kg baggage
const BURN_KG_PER_KG_PAYLOAD = 0.030; // ~3.0% of payload as incremental fuel

export function estimateMarginalSeatCost(input) {
  const fuelPricePerKg = input.fuelPriceCentsPerKg ?? 0;
  const fuel = Math.round(PASSENGER_ALL_UP_KG * BURN_KG_PER_KG_PAYLOAD * fuelPricePerKg);
}
```

shared/src/costing.js

Two words govern how a name may be used, and the difference matters when you review code.

Keyword	Meaning	Use in AMS
const	The name is fixed; the value may not be reassigned.	Used everywhere. A name that never changes is never a surprise.
let	The name is fixed but the value may be reassigned.	Used rarely, only where reassignment is genuinely intended, such as a running total in a loop.
var	The older, function-scoped form.	Never used. It has confusing scope rules and no advantage.

```
let fuelOnBoard = 0; // arriving at stop 0 with empty tanks

for (let i = 0; i < n - 1; i += 1) {
  const price = priceCentsPerKg[i];
  // ... fuelOnBoard genuinely changes here ...
  fuelOnBoard = departure - legFuelKg[i];
}
```

shared/src/routing/refuel.js — let is correct for fuelOnBoard, because it is reassigned each iteration.

Why this matters when you review code

Naming conventions in this codebase: `const` by default, `let` only when a value must change. If you see `var`, it is an error. If you see a `let` that is never reassigned, that is also worth querying — it often signals a design that has drifted.

Numbers are the one thing to fear

Every number in JavaScript that looks ordinary is stored in a format that cannot represent most decimal fractions exactly. The format is called IEEE 754 double precision. It is a hardware standard, deeply optimised for speed, and it is exactly wrong for money.

```
> 0.1 + 0.2
0.30000000000000004

> 0.1 * 3
0.30000000000000004

> 1.005 * 100
100.49999999999999
```

What a JavaScript console shows. These are not printing errors — the stored values genuinely are those numbers.

The error is tiny. It does not matter when you are animating a circle. It matters enormously when you are adding ten million amounts and expecting the total to match an invoice.

The range problem

The other limitation is size. A number can hold integers exactly only up to a certain value. Beyond that, digits are silently lost.

```
// The largest integer a JavaScript number holds exactly:
Number.MAX_SAFE_INTEGER // 9007199254740991 (about 9 quadrillion)

> 9007199254740992 + 1
9007199254740992 // the +1 vanished

> 0.1 + 0.2 === 0.3
false
```

Nine quadrillion cents is ninety billion dollars. That is far more than any airline holds, so the range is not a practical problem for AMS. The important consequence is the *checking* discipline that follows from caring about the limit at all.

```
function assertSafeInteger(v, what = 'amount') {
  if (typeof v !== 'number' || !Number.isSafeInteger(v)) {
    throw new MoneyError(
      `${what} must be a safe integer number of cents; received ${typeof v} ${String(v)}. ` +
      'Floating-point money is forbidden — parse from a string with cents().',
      'MONEY_NOT_SAFE_INTEGER',
    );
  }
}
```

shared/src/money.js — this check runs before essentially every monetary calculation in the system.

What "safe integer" means precisely

A number is "safe" if it is a whole number (no fractional part) and can be represented exactly.

`Number.isSafeInteger` returns true only in that case. A value of 10.5 returns false — not because it is "unsafe" in a security sense, but because it is not an integer and AMS never permits a fractional cent.

Rounding is a decision, not an accident

When a division cannot produce a whole number, JavaScript can return it with a decimal part, or you can ask it to round. Which you choose changes the answer, so AMS writes the rounding explicitly at every point where it happens.

```
Math.floor(x)    // round down
Math.ceil(x)     // round up
Math.round(x)    // round to nearest
Math.trunc(x)    // discard the fractional part
```

These four look interchangeable and are not. `Math.round(-0.5)` gives `-0`, not `1`. `Math.floor(-0.5)` gives `-1`, not `0`. For money, AMS often avoids the built-ins and states the rule itself.

```
// Half-up on the absolute value so the sign is symmetric.
const rounded = numerator >= 0
  ? Math.floor(numerator / denominator + 0.5)
  : -Math.floor(Math.abs(numerator) / denominator + 0.5);
return rounded;
```

shared/src/money.js, applyPpm(). This rounds +2.5 to 3 and -2.5 to -3. The built-in Math.round would give -2 for the second case.

A rounding rule you do not state is a rounding rule you cannot audit

When a finance manager asks "why is this invoice total one cent different from the sum of the lines?", the answer has to be reproducible from the code. "It just rounds" is not an answer. AMS names the rule at the point of use so it can be explained.

Text, and how code reads it

Text in JavaScript is called a string. Strings are written in quotes, and the quotes do not appear in the value.

```
const text = 'finance_treasury';
const len = text.length;          // 17
const upper = text.toUpperCase(); // 'FINANCE_TREASURY'
const part = text.slice(0, 7);    // 'finance'
const joined = [1, 2, 3].join('-'); // '1-2-3'
const padded = '7'.padStart(3, '0'); // '007'
```

Template literals are the modern way to build a string. They use backticks and allow a value to be inserted directly.

```
throw new MoneyError(
  `Monetary amount out of safe integer range: "${input}"`,
  'MONEY_OVERFLOW',
);
```

shared/src/money.js. `${input}` inserts the value of the variable named `input`. Note the deliberate quotes around it, so the reader can see exactly what was received.

Regular expressions: matching a pattern

A regular expression is a compact description of a shape of text. AMS uses them where hand-written checking would be long and error-prone.

```
const cleaned = raw.replace(/[\s, _]/g, '');
const match = /^(?)(\d*)(?:\.(\d{1,2}))?$/ .exec(cleaned);
if (!match) throw new MoneyError(`Cannot parse monetary amount: "${input}"`, 'MONEY_UNPARSEABLE');

const [, sign, whole = '0', frac = ''] = match;
```

shared/src/money.js — parsing "1234.56". Read it as: optionally a minus sign, then some digits, then optionally a dot and one or two digits, and nothing else.

Symbol	Means	In this pattern
^	start of the text	the value must begin here
\$	end of the text	and end there
-?	a minus sign, optional	captured as group 1
\d*	zero or more digits	group 2, the whole amount
\.?	a dot, optional	inside a non-capturing group
\d{1,2}	one or two digits	group 3, the cents
...	the remainder of the code	skipped deliberately

```
const [, sign, whole = '0', frac = ''] = match;
```

The leading comma is a hole: the first item (the whole match) is skipped, and the three named values are taken from positions 1, 2 and 3. = '0' supplies a default when a group did not participate in the match.

Why not use Number() to parse money?

Number("1234.56") gives a float, and the fraction is inexact. The whole point of the regex is to keep the digits as *text* — the characters "1234" and "56" — and only then combine them into an integer. No fractional value is ever created.

Decisions: if, and the shape of a condition

```
if (ad.superseded) {
  return { applies: false, reason: 'Superseded by a later directive', complianceStatus: null };
}
if (ad.aircraftType !== aircraft.aircraftType) {
  return { applies: false, reason: `Not applicable to ${aircraft.aircraftType}`, complianceStatus: null };
}
```

shared/src/airworthiness.js. Each condition tests one thing and then returns immediately.

That early-return style is used deliberately and consistently. The alternative — a long chain of `else if` — becomes unreadable past about three branches, and in a system that decides whether an aircraft may fly, readability is a safety property.

```
let verdict;
if (agg.flights === 0) verdict = 'NO_DATA';
else if (agg.contributionCents < 0) verdict = 'LOSS_MAKING';
else if (agg.loadFactorPpm < targetLoadFactorPpm) verdict = 'MARGINAL';
else verdict = 'PROFITABLE';
```

shared/src/costing.js — a verdict chain. Order matters: the first matching condition wins, so "no data" must be checked before "loss making".

Comparison operators

Written	Means	Common mistake
=	assign a value	using it to compare; this is the single most common JavaScript bug
==	compare, with type conversion	<code>0 == "0"</code> is true; AMS never uses it
===	compare, type must match too	always used in AMS
!==	the negation of ===	always used in AMS
< <= > >=	order comparison	works on strings by alphabetical order, which surprises people

```
if (commitment.amount_cents !== amountCents) {
```

This is the single most important line of JavaScript to internalise as a non-programmer, because it is the one that appears most often in accidents. `=` sets something. `==` is a discouraged loose comparison. `===` asks "are these exactly the same, type included". In AMS you will see `===` and `!==` in every conditional, and you should expect to see nothing else.

The ternary operator

```
bufferPpm = ctx.expectedBurnVariancePpm ?? 30_000; // default +3%

varianceDirection: varianceKg > 0 ? 'OVER' : varianceKg < 0 ? 'UNDER' : 'ON_PLAN',

const symbol = { USD: '$', EUR: '€', GBP: '£', ZWG: 'ZWG ' }[currency] ?? `${currency}`;
```

condition ? valueIfTrue : valueIfFalse. Used where it is short and clear, avoided where it would need a nested chain.

Loops, and when not to use one

```

for (let i = 0; i < n - 1; i += 1) {
  const price = priceCentsPerKg[i];
  // ... do work ...
}

for (const entry of Object.entries(lines)) {
  if (FLIGHT_ATTR.has(category)) byCategory.FLIGHT_ATTRIBUTABLE += amount;
}

```

Two loop forms. The first counts. The second visits each element of a collection.

```

flights.reduce(
  (acc, f) => ({
    flights: acc.flights + 1,
    asks: acc.asks + f.asks,
    rpks: acc.rpks + f.rpks,
    revenueCents: acc.revenueCents + f.revenueCents,
  }),
  { flights: 0, asks: 0, rpks: 0, revenueCents: 0 },
);

```

shared/src/costing.js — reduce folds a list into a single value. The first argument is a function; the second is the starting value.

reduce is the most powerful and least readable tool in this codebase. It appears almost exclusively in one situation: summing a group of figures into a total. Where AMS uses it, it is because the alternative would be a loop that builds an object field by field — which is more code and more room for a mistake.

Reading a reduce without reading JavaScript

The pattern is always the same. The function receives two things: `acc`, which is everything computed so far, and `f`, the current element. It returns the new value of `acc`. The second argument is where counting starts. Read it as: "starting from zero, for each flight, add its figures to the running total."

Functions: naming a piece of work

A function is a named, reusable piece of work. AMS uses functions as the primary unit of organisation, and each one is expected to do a single well-defined thing.

```

/**
 * Distribute `total` across `weights` so that the parts sum to EXACTLY
 * the total, using the largest-remainder method.
 *
 * This is the single most important function in the costing engine.
 * When you split a monthly fuel cost across 400 flights you cannot
 * round each flight's share independently: 400 x 1 cent of rounding
 * drift becomes 4 dollars that cannot be reconciled to the supplier
 * invoice. Largest-remainder guarantees zero leakage.
 *
 * @param {Cents} total must be >= 0
 * @param {number[]} weights non-negative
 * @returns {Cents[]} same length as weights, sums to total
 */
export function allocate(total, weights) {

```

shared/src/money.js. The comment above the function explains not what it does but why it must exist that way.

That comment convention is worth understanding, because it is a deliberate cultural choice in this codebase. A comment that restates the code is noise. A comment that explains *why this approach rather than the obvious one* prevents the next engineer from "simplifying" it into a bug.

Arguments, and the ones you cannot do without

```
export function cents(input) { ... }

export function sum(...values) { ... }

export function assertReconciles(parts, expected, context = 'reconciliation') { ... }
```

Form	What it means	Example
cents(input)	one required argument	the amount to parse
sum(...values)	any number of arguments, gathered into an array	sum(1, 2, 3)
context = 'reconciliation'	optional, with a default	used if the caller omits it

Returning more than one value

```
export function solveRoute({ flights, source, destination, tankCapacityKg, minReserveKg }) {
  // ...
  return {
    feasible: true,
    costCents: label.costCents,
    fuelUsedKg: label.fuelUsedKg,
    stops: reconstructStops(label),
    labelsExpanded, labelsPruned, maxFrontier,
  };
}
```

JavaScript has one return value, so functions that produce several related facts return a single object. Callers then read the fields by name.

```
const result = solveRoute({ flights, source, destination, tankCapacityKg, minReserveKg });
result.feasible // true
result.costCents // 112000
result.stops // ['JFK', 'YUL', 'GRU']
```

The destructured parameter deserves a note: { flights, source, ... } means "the caller passes one object, and I take these named values out of it". This avoids a long list of arguments that must be passed in the right order.

Objects: labelled collections

```

/**
 * @typedef {Object} FuelPlan
 * @property {number} taxiFuelKg
 * @property {number} tripFuelKg
 * @property {number} contingencyKg
 * @property {number} alternateKg
 * @property {number} finalReserveKg   Regulatory minimum landing reserve
 * @property {number} [etopsReserveKg] Only on ETOPS routes
 * @property {number} upliftKg        Total taken on board
 * @property {number} priceCentsPerKg Realised uplift price
 */

```

shared/src/fuel.js. This describes the shape of a fuel plan: what fields exist, what type each is, and which are optional. The comment after each field is a business note, not a technical one.

```

const plan = {
  taxiFuelKg: 250,
  tripFuelKg: 7800,
  contingencyKg: 300,
  alternateKg: 620,
  finalReserveKg: 1500,
  upliftKg: 10470,
  priceCentsPerKg: 95,
};

plan.tripFuelKg          // 7800
plan.etopsReserveKg      // undefined – never provided
plan.etopsReserveKg ?? 0 // 0 – substitute when absent

```

The difference between absent and zero

This is subtle and it matters. A field that was never provided is undefined. A field explicitly set to zero is 0. They behave differently in a check.

```

if (plan.etopsReserveKg !== undefined) { ... } // present, even if 0
if (plan.etopsReserveKg) { ... }              // truthy: skips 0

const etops = plan.etopsReserveKg ?? 0; // 0 only when undefined
const zero  = plan.etopsReserveKg || 0; // 0 whenever falsy

```

Why AMS prefers ?? in financial code

?? substitutes a default only when the value is absent or null. || substitutes whenever the value is anything falsy — including a legitimate zero. In a ledger where zero is a meaningful amount, collapsing zero into a default can hide a real data problem. AMS uses ?? where zero carries meaning.

Spreading and merging

```

const required = 3200 + 8600 + 1500;

const required = i === n - 1
  ? reserveKg
  : Math.max(reserveKg, required[i + 1]);
required[i] = legFuelKg[i] + neededBeyond;

return {
  ...totals,
  loadFactorPpm: ...,
  caskMicrocents: caskMicrocents(totals.operatingCostCents, totals.asks),
};

```

Three spreads in one sample. `...totals` copies every field of the `totals` object into the new one; then specific fields overwrite or add to them.

Arrays: ordered collections

```

export const FLIGHT_ATTRIBUTABLE_CATEGORIES = [
  'fuel', 'crew_flight', 'crew_cabin', 'crew_training', 'maintenance_direct',
  'maintenance_reserve', 'airport_charges', 'navigation_charges', 'catering',
  'handling', 'de_icing',
];

```

`shared/src/domain.js`. This is the single source of truth for which costs count as variable with the flight.

The array methods AMS relies on, and what each does:

Method	What it returns	In AMS
<code>.map(f)</code>	a new array of the same length, each element passed through <code>f</code>	convert cost lines into category totals
<code>.filter(p)</code>	a new array containing only elements where <code>p</code> is true	keep only applicable directives
<code>.reduce(f, start)</code>	a single accumulated value	sum a list into a total
<code>.find(p)</code>	the first element where <code>p</code> is true	locate a specific budget line
<code>.includes(v)</code>	true if the value is present	check a driver name is permitted
<code>.sort(cmp)</code>	the array, reordered in place	rank deadlines; always supply a comparator
<code>.at(-1)</code>	the last element	the most recent entry in an audit chain

The one array trap worth knowing about

`sort()` with no argument sorts alphabetically, so `[100, 9, 10]` becomes `[10, 100, 9]`. It also changes the original array rather than a copy. AMS passes an explicit comparator and, where the original must survive, copies first: `[...flights].sort(...)`.

```
const scoped = allEntries
  .filter((e) => e.tenantId === tenantId)
  .sort((a, b) => a.id - b.id);

const ordered = [...lines].sort((a, b) => a.lineId.localeCompare(b.lineId));
```

backend/src/audit/chain.js and cost-authority.js. Read the comparator (a, b) => a.id - b.id as: "if a comes before b, return a negative number; if after, positive".

The special values undefined, null, and NaN

JavaScript has several values that mean "no value", and they are not interchangeable. Precision here directly affects correctness.

Value	Means	Arises when
undefined	nothing was provided	a field omitted, a function with no return, an array past its end
null	deliberately empty	the code sets it, usually to record "no applicable deadline"
NaN	the result of a calculation that has no meaning	0 / 0, Math.sqrt(-1), parseInt("abc")

```
const target = best.get(destination);
if (!target || target.size === 0) return EMPTY;

nextDue: [...overdue, ...dueSoon].sort((a, b) => a.remainingPpm - b.remainingPpm)[0] ?? null,
```

shared/src/airworthiness.js. The first line handles the case where nothing was found at all; the second records an explicit "none" rather than leaving it undefined.

In AMS, null is used where a missing value is a meaningful business fact — no next deadline exists, no directive applies. undefined is used where information simply was not supplied. The distinction lets a reviewer tell "we looked and found nothing" from "nobody looked".

Modules: how files talk to each other

Each AMS file is a module. A module exports some things and keeps the rest private. Nothing outside can reach a private helper.

```
import { allocate, assertReconciles, MoneyError, sum } from './money.js';
import { flightMetrics } from './metrics.js';
import {
  ALLOCATION_DRIVERS,
  FLIGHT_ATTRIBUTABLE_CATEGORIES,
  FLEET_FIXED_CATEGORIES,
  PERIOD_CATEGORIES,
} from './domain.js';
```

shared/src/costing.js — the top of the file. Three imports: named values from specific files.

```
import { createHash } from 'node:crypto';

/** @typedef {Cents} Cents Integer minor units. Never a float result. */

export function cents(input) { ... } // exported: anyone may use it

function assertSafeInteger(v, what = 'amount') { ... } // private to this file
```

```
const FLIGHT_ATTR = new Set(FLIGHT_ATTRIBUTABLE_CATEGORIES);
const FLEET_FIXED = new Set(FLEET_FIXED_CATEGORIES);
const PERIOD = new Set(PERIOD_CATEGORIES);

for (const [category, amount] of Object.entries(lines)) {
  if (FLIGHT_ATTR.has(category)) byCategory.FLIGHT_ATTRIBUTABLE += amount;
  else if (FLEET_FIXED.has(category)) byCategory.FLEET_FIXED += amount;
  else if (PERIOD.has(category)) byCategory.PERIOD += amount;
  else byCategory.UNKNOWN += amount;
}
```

shared/src/costing.js. Arrays become Sets once at the top of the file, then membership is tested directly. Converting an array to a Set for repeated has checks is a standard optimisation, and it also reads closer to intent: "is this a flight-attributable category?"

The one thing to notice about imports

Every import ends with `.js`. The full filename is written out. This is deliberate and is what makes the code work directly in Node with no bundler and no configuration. Nothing is inferred, so nothing can be guessed wrong.

The dependency direction

AMS has a strict one-way dependency structure, and it is the reason the money rules cannot be bypassed.

```
backend/src/modules/finance/cost-authority.js
  ■■■ imports shared/src/money.js          (may use money)

shared/src/costing.js
  ■■■ imports shared/src/money.js          (may use money)

shared/src/money.js
  ■■■ imports nothing                      (the foundation)
```

Nothing imports `money.js` *into* it. It is the base of the dependency graph. This is what makes "the only module permitted to do arithmetic on money" a fact about the architecture rather than a promise in a comment.

Part III — Money, the discipline that defines this codebase

If you read one part of this guide, read this one. Almost every design decision in AMS traces back to the requirement that money must be exact.

Why nobody uses decimals for money in code

Every general-purpose programming language has this problem, because computer hardware stores numbers in binary and 0.1 has no finite binary representation. The usual workaround is to hold money as a whole number of minor units.

```
// The same amount, three ways
const a = 0.1 + 0.2;           // 0.30000000000000004
const b = 10 + 0.2;           // 10.2   (looks fine)
const c = 1.1 * 3;             // 3.3000000000000003

// Add a penny ten million times
let total = 0;
for (let i = 0; i < 10_000_000; i += 1) total += 0.01;
total // 99999.99999999999 - 1/100 of a cent adrift
```

The error is individually invisible. It only becomes visible in aggregate, which is exactly when you least want it: a trial balance that refuses to foot, or an invoice total that differs from the sum of its lines by a few cents.

Cents: the integer unit AMS chose

```
/**
 * AMS — Integer money arithmetic.
 *
 * THE ONLY MODULE PERMITTED TO PERFORM ARITHMETIC ON MONETARY VALUES.
 *
 * Rule: every monetary amount is an integer number of minor units
 * (US cents). Floating point is never used for money anywhere in AMS.
 *
 * Why this matters concretely, not in theory:
 *
 *   0.1 + 0.2 !== 0.3           // IEEE-754 binary representation error
 *
 * In a ledger, that error compounds across a million transactions and
 * the trial balance stops footing. Under IFRS the ledger must balance
 * exactly. The defence is structural: the type system has no way to
 * represent a fractional cent, and the one function that parses money
 * from a string uses integer arithmetic to do it.
 *
 * @module money
 */
```

shared/src/money.js — the file header. Everything the team needs to know about the money rule is stated before a single function.

The key architectural point is in that third paragraph: the defence is *structural*. It is not a code review instruction or a lint rule. There is simply no function in AMS that accepts a fractional cent, because every path into a monetary value passes through a check.

```
function assertSafeInteger(v, what = 'amount') {
  if (typeof v !== 'number' || !Number.isSafeInteger(v)) {
    throw new MoneyError(
      `${what} must be a safe integer number of cents; received ${typeof v} ${String(v)}. ` +
      'Floating-point money is forbidden – parse from a string with cents().',
      'MONEY_NOT_SAFE_INTEGER',
    );
  }
}

export function add(a, b) { assertSafeInteger(a); assertSafeInteger(b); return a + b; }
export function subtract(a, b) { assertSafeInteger(a); assertSafeInteger(b); return a - b; }

export function multiplyWhole(value, factor) {
  assertSafeInteger(value);
  if (!Number.isInteger(factor)) {
    throw new MoneyError(`Multiplier must be a whole number; received ${factor}`, 'MONEY_NON_INTEGER_FACTOR');
  }
  return value * factor;
}
```

shared/src/money.js. Even addition refuses to proceed unless both operands are whole cents.

Why multiplyWhole exists at all

Money must sometimes be multiplied — by a quantity of seats, a number of hours, a rate. But "multiply a cent amount by 1.5" has no meaning in a ledger; it produces half a cent. The function therefore permits only whole-number multipliers. If you need a fractional result, you have a percentage, and there is a separate function for that (below). This makes the illegal operation impossible to express rather than merely discouraged.

Parsing a human amount safely

```

export function cents(input) {
  const raw = typeof input === 'number' ? input.toFixed(2) : String(input);
  const cleaned = raw.replace(/[\s, _]/g, '');
  const match = /^(?)(\d*)(?:\.\d{1,2})?$/i.exec(cleaned);
  if (!match) throw new MoneyError(`Cannot parse monetary amount: "${input}"`, 'MONEY_UNPARSEABLE');

  const [, sign, whole = '0', frac = ''] = match;
  if (whole === '' && frac === '') {
    throw new MoneyError(`Cannot parse monetary amount: "${input}"`, 'MONEY_UNPARSEABLE');
  }
  // Pad/truncate fractional part to exactly 2 digits.
  const frac2 = (frac + '00').slice(0, 2);
  const value = Number(whole || '0') * 100 + Number(frac2);
  const signed = sign === '-' ? -value : value;
  if (!Number.isSafeInteger(signed)) {
    throw new MoneyError(`Monetary amount out of safe integer range: "${input}"`, 'MONEY_OVERFLOW');
  }
  return signed;
}

```

shared/src/money.js — the single point at which human text becomes money in AMS.

Line by line, for a reader who does not write code:

1. **Decide how to read the input.** If a number was passed, it is converted to text first with two decimal places. If text was passed, it is used as-is. Working in text is the whole point.
2. **Remove cosmetic characters.** Spaces, commas and underscores are stripped, so "1, 234.56" and "1 234.56" and "1234.56" all parse identically. This is real-world tolerance for how people type.
3. **Match the pattern.** Only an optional minus sign, digits, and optionally one or two decimal digits. Anything else — a third decimal place, letters, an exponent — is rejected outright.
4. **Reject empty matches.** "." or "-" alone must not become zero.
5. **Pad the fraction.** "7" becomes "07", so 7 dollars is 700 cents rather than 7.
6. **Combine as integers.** `Number("1234") * 100 + Number("56")` gives 123456. No fractional value exists at any point.
7. **Check the range.** If the result exceeds the safely-representable limit, throw rather than silently losing digits.

```

cents('1234.56')      // 123456
cents('1,234.56')     // 123456
cents('1 234.56')     // 123456
cents('-78.90')       // -7890
cents('.50')          // 50
cents('7')            // 700
cents('12.5')         // 1250
cents('12.345')       // throws — three decimal places
cents('abc')          // throws
cents('')              // throws

```

Rendering money back to text

```
export function format(value, currency = 'USD') {
  assertSafeInteger(value);
  const negative = value < 0;
  const abs = Math.abs(value);
  const whole = Math.floor(abs / 100);
  const frac = abs % 100;
  const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
  const symbol = { USD: '$', EUR: '€', GBP: '£', ZWG: 'ZWG ' }[currency] ?? `${currency} `;
  return `${negative ? '-' : ''}${symbol}${grouped}.${String(frac).padStart(2, '0')}`;
}
```

shared/src/money.js.

Note that this does no arithmetic on a fraction. The whole part and the cents part are separated by `Math.floor` and the remainder operator `%`, both of which are exact for integers. The formatting regex inserts thousands separators. A cents value of 5 renders as `".05"`, never as `".5"`.

Rounding: the rules AMS applies

Rounding is unavoidable in real finance: you cannot invoice half a cent. What matters is that the rule is stated and applied uniformly.

```
// Fuel is the classic integer-arithmetic trap. fuelKg x priceCentsPerKg
// is fractional. We must round ONCE, here, at the point the fact is
// created – not per-line later, and never by float.
const fuelCents = Math.round(fuelKg * fuelPricePerKg);
if (!Number.isSafeInteger(fuelCents)) {
  throw new MoneyError('Fuel cost overflow – split the uplift across delivery notes', 'COST_FUEL_OVERFLOW');
}
```

shared/src/costing.js. The comment states the policy: round once, at the point the fact is created.

Why "round once, at the point of creation"

The alternative is to keep exact fractions throughout and round at the end. That is often more accurate in isolation, but it forces every downstream consumer to handle fractional cents — and any consumer that forgets becomes the source of a discrepancy. Rounding once, early, means everything downstream works with values that are already real.

Half-up, symmetric about zero

```
export function applyPpm(value, ratePpm) {
  assertSafeInteger(value);
  if (!Number.isInteger(ratePpm)) {
    throw new MoneyError(`Rate must be integer ppm; received ${ratePpm}`, 'MONEY_NON_INTEGER_RATE');
  }
  const numerator = value * ratePpm;
  const denominator = 1_000_000;
  // Half-up on the absolute value so the sign is symmetric.
  const rounded = numerator >= 0
    ? Math.floor(numerator / denominator + 0.5)
    : -Math.floor(Math.abs(numerator) / denominator + 0.5);
  return rounded;
}
```

shared/src/money.js. This applies a rate expressed in parts per million, used for currency conversion, fuel escalators and indexation.

The comment "half-up on the absolute value so the sign is symmetric" describes a real problem. JavaScript's built-in `Math.round` rounds 2.5 to 3 but -2.5 to -2. Applying the rule to the magnitude and then restoring the sign gives -3, which is what a finance team expects when they say "round half up".

Splitting money without losing a cent

This is the single most important function in the costing engine, and it deserves a full explanation.

```
export function allocate(total, weights) {
  assertSafeInteger(total, 'total');
  if (total < 0) throw new MoneyError('Cannot allocate a negative total', 'MONEY_NEGATIVE_TOTAL');
  if (weights.length === 0) {
    if (total !== 0) throw new MoneyError('Cannot allocate non-zero total across zero weights', 'MONEY_NO_WEIGHTS');
    return [];
  }
  if (weights.some((w) => !Number.isFinite(w) || w < 0)) {
    throw new MoneyError('Allocation weights must be finite and non-negative', 'MONEY_BAD_WEIGHT');
  }

  const weightSum = weights.reduce((a, b) => a + b, 0);
  if (weightSum === 0) {
    if (total !== 0) {
      throw new MoneyError('Cannot allocate non-zero total across zero total weight', 'MONEY_ZERO_WEIGHT_SUM');
    }
    return weights.map(() => 0);
  }

  const rawShares = weights.map((w) => (total * w) / weightSum);
  const floors = rawShares.map((s) => Math.floor(s));
  let remainder = total - floors.reduce((a, b) => a + b, 0);

  // Hand the leftover cents to the largest fractional parts, biggest first.
  const order = rawShares
    .map((share, index) => ({ index, frac: share - Math.floor(share) }))
    .sort((a, b) => b.frac - a.frac || a.index - b.index);

  const result = [...floors];
  for (let i = 0; i < order.length && remainder > 0; i += 1, remainder -= 1) {
    result[order[i].index] += 1;
  }
  return result;
}
```

shared/src/money.js — largest-remainder allocation.

The business problem

A fuel supplier invoices US\$48,391.55 for a month. That amount must be shared across 400 individual flights in proportion to the fuel each one burned, so that each flight carries a defensible cost.

```
// Naive: divide, then round each one.
4839155 / 400 = 12097.8875 cents
// Rounded independently: 12098 cents each
// 12098 x 400 = 4839200 cents - 45 cents MORE than the invoice.

const allocate = allocate(4839155, weights);
sum(...allocate) === 4839155; // true, always
```

Forty-five cents is not a large amount. But the same logic applies to every allocation in the system, across every month, across every aircraft and route. Errors of this size are individually tolerable and collectively fatal to reconciliation: a supplier invoice that can never be tied to the sum of its component entries is an invoice the airline cannot close.

The method, step by step

1. **Work out each share exactly**, as a fraction. Do not round yet.
2. **Take the whole part of each share** — round every one *down*. The sum is now guaranteed to be less than or equal to the total.
3. **Count the leftover cents**. This is the difference between the floored sum and the original total. It cannot be more than one per share.
4. **Rank the shares by how much they lost** in the previous step — the largest fractional part lost the most.
5. **Hand out the leftover cents**, one each, largest-loss first.
6. **Break ties by position**, so the same input always produces the same output. This matters for audit reproducibility.

Why determinism is a requirement, not a nicety

Two engineers running the same allocation on the same data must get the same answer, or the audit trail will show a figure that changed with no underlying event. That is why the tie-break is explicit: `.sort((a, b) => b.frac - a.frac || a.index - b.index)`.

Percentages as integers

A percentage such as 17.5% cannot be stored as a fraction without introducing the original problem. AMS stores rates as parts per million.

```
// 17.5% stored as 175_000 parts per million
// 0.175% stored as 17_500 parts per million
const ratePpm = 175_000;
applyPpm(1_000_00, ratePpm); // 17500 cents = $175.00 exactly

export const COMPLIANCE_WARNING_THRESHOLD_PPM = 50_000; // 5% of remaining life
const bufferPpm = ctx.expectedBurnVariancePpm ?? 30_000; // default +3%
const cents = fee * (1_000_000 + VAT_PPM) / 1_000_000;
```

Parts per million give six decimal places of precision while staying a whole number. They appear throughout AMS: load factors as ppm, margins as ppm, compliance thresholds as ppm. When you see a

field name ending _Ppm, it is a rate stored as an integer.

Part IV — Measuring an airline

The airline industry expresses almost every commercial decision in a handful of ratios. This part explains them and shows how AMS computes them exactly.

The industry ratios, explained

```
/**
 * AMS — Traffic and unit-economics metrics.
 *
 * The airline industry's entire commercial vocabulary reduces to a
 * handful of ratios. This module computes them exactly, in integer
 * cents per unit, so that two carriers' CASK figures are actually
 * comparable — which they are not, in practice, unless the numerator
 * is built from the same cost taxonomy (see domain.js COST_CATEGORIES).
 *
 * Key definitions, per IATA:
 *   ASK  Available Seat Kilometres — seats offered x distance flown.
 *        The supply side. Unaffected by whether anyone buys a ticket.
 *   RPK  Revenue Passenger Kilometres — paying passengers x distance.
 *        The demand side, and the only passenger-capacity measure that
 *        also carries revenue.
 *   PLF  Passenger Load Factor = RPK / ASK.
 *
 *   CASK  Cost per Available Seat Kilometre = operating cost / ASK
 *   RASK  Revenue per Available Seat Kilometre = revenue / ASK
 *
 *   CASM / RASM are the same ratios using statute miles (ASM). They are
 *   provided for US-filed comparability. 1 statute mile = 1.609344 km.
 *
 * Break-even load factor is the CASK/RASK ratio: the load factor below
 * which the flight does not cover its own variable and allocated cost.
 * The whole of schedule planning is an exercise in raising load factor
 * or lowering CASK on routes where their ratio is uncomfortable.
 *
 * @module metrics
 */
```

shared/src/metrics.js — the module header. Read this twice; it is the clearest statement in the codebase of what the system measures and why.

Asking for capacity: ASK and RPK

```
export function ask(seatsOffered, distanceKm) {
  assertNonNegativeInt(seatsOffered, 'seatsOffered');
  assertNonNegativeInt(distanceKm, 'distanceKm');
  return seatsOffered * distanceKm;
}

export function rpk(passengers, distanceKm) {
  assertNonNegativeInt(passengers, 'passengers');
  assertNonNegativeInt(distanceKm, 'distanceKm');
  return passengers * distanceKm;
}
```

shared/src/metrics.js

A concrete example. A 189-seat aircraft flies 4,000 km. It offers 756,000 seat-kilometres of capacity regardless of whether anyone buys a ticket. If it carries 151 paying passengers, those passengers consume 604,000 revenue passenger-kilometres. The load factor is the second divided by the first: 79.9%.

```
export function loadFactorPpm(seatsOffered, passengers, distanceKm) {
  const a = ask(seatsOffered, distanceKm);
  const r = rpk(passengers, distanceKm);
  if (a === 0) return 0;
  return Math.floor((r / a) * 1_000_000 + 0.5);
}
```

shared/src/metrics.js — 799_000 ppm is 79.9%.

Why the result is not 0.799

Because 0.799 is a fraction that cannot be represented exactly, and because the whole problem of Part III returns. Storing the load factor as 799000 — an integer number of parts per million — means it can be stored in a database, summed across flights, and compared exactly, with no drift.

Cost and revenue per unit: CASK and RASK

```
export function caskMicrocents(operatingCostCents, asks) {
  if (!Number.isSafeInteger(operatingCostCents)) throw new MoneyError('cost must be integer cents', 'METRIC_BAD_C');
  assertNonNegativeInt(asks, 'asks');
  if (asks === 0) return 0;
  // cost cents * 1e4 -> microcents, then per ASK
  return Math.floor((operatingCostCents * 1_000_000) / asks + 0.5);
}
```

shared/src/metrics.js

CASK is the industry's most-quoted efficiency number: the cost of providing one seat-kilometre of empty capacity. Lower is better. RASK is the revenue from that same seat-kilometre, and the gap between them is the margin.

```
// Microcents, not cents
const cask = caskMicrocents(2_100_000, 756_000); // 2777 microcents
// = 2.777 US cents per seat-km, the industry norm for a narrowbody

export const KM_PER_STATUTE_MILE = 1_609_344; // scaled by 1e6 to stay integer
```

The unit is microcents — cents multiplied by ten thousand — for a concrete reason stated in the code comment:

```
/**
 * Returned in microcents (cents x 10,000) rather than cents because a
 * long-haul flight spreads cost over hundreds of millions of ASK; a
 * plain cents figure would round to zero and destroy the signal.
 * Microcents are still an exact integer.
 */
```

The general lesson: choose a unit that cannot round to zero

If you divide a large cost by a very large distance in cents, the answer may fall below one cent and become 0 — a number that looks like "free" and is meaningless. Scaling the unit preserves the signal while keeping the value a whole number. AMS applies this in several places: microcents for CASK, parts per million for ratios, milli-units for hours and weights.

Why you must never average an average

This is the most important modelling rule in the module, and the code comment states it plainly.

```
/**
 * Roll flights up into a route, fleet-type or period aggregate.
 * Aggregation of ratios is deliberately NOT the average of the parts –
 * it is always recomputed from the summed numerators and denominators.
 * Averaging per-flight load factors is a classic modelling error: it
 * weights a short hop equally with a long haul.
 *
 * @param {Array<FlightInput>} flights
 */
```

shared/src/metrics.js

Concretely. Consider a two-flight period:

Flight	Distance	Passengers	Seats	ASK	RPK	Load factor
Short hop	800 km	170	189	151,200	136,000	90.0%
Long haul	9,000 km	150	189	1,701,000	1,350,000	79.4%
Average of the two						84.7%
Correct total	9,800 km	320	378	1,852,200	1,486,000	80.2%

The average of the two load factors is 84.7%. The true figure is 80.2%. The short hop carried 90% of its seats, but it represents only 8% of the distance, so its good performance barely counts. Averaging gives it equal weight with a flight twelve times longer.

```

const totals = flights.reduce(
  /** @returns {RollUpTotals} */
  (acc, f) => {
    const m = flightMetrics(f);
    acc.asks += m.asks;
    acc.rpks += m.rpks;
    acc.revenueCents += m.revenueCents;
    acc.operatingCostCents += m.operatingCostCents;
    // ...
    return acc;
  },
  /** @type {RollUpTotals} */ ({
    asks: 0, rpks: 0, revenueCents: 0, operatingCostCents: 0,
    seatsOffered: 0, passengers: 0, blockHoursMilli: 0, distanceKm: 0, sectors: 0,
  }),
);

return {
  ...totals,
  loadFactorPpm: totals.asks === 0 ? 0 : Math.floor((totals.rpks / totals.asks) * 1_000_000 + 0.5),
  caskMicrocents: caskMicrocents(totals.operatingCostCents, totals.asks),
};

```

shared/src/metrics.js — sum the ingredients, divide once at the end.

A subtle JavaScript trap this code avoids

Notice that the accumulator is written as `acc.asks += m.asks` and then `return acc` — the accumulator is *mutated* rather than replaced. The codebase's comment on this says that `Array.reduce` otherwise infers the accumulator type from the element type, which silently widens it and loses every aggregate field. Both styles work at runtime; the explicit type annotation is what makes the check gate reliable.

Break-even load factor

```

export function breakEvenLoadFactorPpm(operatingCostCents, revenueCents) {
  if (!Number.isSafeInteger(operatingCostCents) || !Number.isSafeInteger(revenueCents)) {
    throw new MoneyError('cost and revenue must be integer cents', 'METRIC_BAD_INPUT');
  }
  if (revenueCents === 0) return Number.MAX_SAFE_INTEGER; // never breaks even
  return Math.floor((operatingCostCents / revenueCents) * 1_000_000 + 0.5);
}

```

shared/src/metrics.js

Break-even load factor answers one question: at what occupancy does the flight exactly cover its cost? It is the ratio of cost to revenue, because the distance cancels out.

```
// breakEvenLF = CASK / RASK = cost/ASK / (revenue/ASK) = cost/revenue
```

The line `if (revenueCents === 0) return Number.MAX_SAFE_INTEGER` is a good example of defensive design. A flight that carried nobody has no revenue, so the ratio is undefined. Rather than divide by zero and produce `Infinity` or `NaN` — both of which would silently poison every later calculation — the function returns the largest possible integer, which correctly reads as "this flight never breaks even".

Why Number.MAX_SAFE_INTEGER and not Infinity

Infinity is a valid JavaScript number but an invalid database value and an invalid JSON value. Returning a real integer means the figure can be stored, compared and rendered without special cases downstream.

```
export function utilisationMilli(blockHours, aircraftDays) {  
  if (aircraftDays <= 0) return 0;  
  return Math.floor((blockHours / aircraftDays) * 1000 + 0.5);  
}
```

shared/src/metrics.js — utilisation in hours per aircraft per day, stored with three decimal places of precision.

The comment calls this "the single strongest driver of unit cost in the industry". An aircraft on the ground earns nothing and still burns money. Utilisation is therefore the operational metric with the largest effect on CASK.

Part V — The costing engine

This is the module the whole platform hangs off. It answers the question an airline most needs answered and least often can: what did this flight actually cost?

Two kinds of cost

```
* Cost falls into two structural buckets:
*
*   FLIGHT-ATTRIBUTABLE varies directly with flying this sector:
*   fuel, duty crew, landing fees, handling, catering.
*   Known on the day of flight.
*
*   FLEET-FIXED         fixed for the period regardless of whether a
*   particular aircraft flies:
*   lease, insurance, spares holding, admin crew,
*   distribution. Allocated to flights by an
*   explicit driver — never by accident.
*
* The distinction is not accounting pedantry. It determines whether a
* route is viable at all. A long-haul sector might have a marginal
* fuel cost of $4,800 and a full cost of $19,000; pricing to marginal
* cost fills the plane but loses money, and pricing to full cost loses
* the traffic. Both numbers must exist.
```

shared/src/costing.js — the module header, which is the best explanation of airline cost behaviour anywhere in this codebase.

The distinction is real and it has a direct commercial consequence. Consider a 12-hour sector costing US\$19,000 in total. Flying it with five extra passengers adds almost nothing — fuel for a few kilograms of extra weight, a few trays. The landing fee does not change. The crew does not change. The lease certainly does not change.

Approach	What it tells you	What happens
Price to marginal cost	what each extra seat genuinely costs to carry	The plane fills, and the airline still loses money
Price to full cost	what the flight needed in total, including the share of lease and insurance	The flight pays for itself, but the fare loses the market

Building the cost of one sector

```

export function buildFlightCost(input) {
  if (!Number.isInteger(input.distanceKm) || input.distanceKm < 0) {
    throw new MoneyError('distanceKm must be a non-negative integer', 'COST_BAD_DISTANCE');
  }

  const fuelKg = input.fuelKg ?? 0;
  const fuelPricePerKg = intOr(input.fuelPriceCentsPerKg, 0, 'fuelPriceCentsPerKg');
  if (fuelKg < 0) throw new MoneyError('fuelKg cannot be negative', 'COST_NEGATIVE_FUEL');

  // Fuel is the classic integer-arithmetic trap. fuelKg x priceCentsPerKg
  // is fractional. We must round ONCE, here, at the point the fact is
  // created – not per-line later, and never by float.
  const fuelCents = Math.round(fuelKg * fuelPricePerKg);
  if (!Number.isSafeInteger(fuelCents)) {
    throw new MoneyError('Fuel cost overflow – split the uplift across delivery notes', 'COST_FUEL_OVERFLOW');
  }

  /** @type {Record<string, number>} */
  const lines = {
    fuel: fuelCents,
    crew_flight: intOr(input.flightCrewCents, 0, 'flightCrewCents'),
    crew_cabin: intOr(input.cabinCrewCents, 0, 'cabinCrewCents'),
    crew_training: intOr(input.crewTrainingCents, 0, 'crewTrainingCents'),
    maintenance_direct: intOr(input.maintenanceDirectCents, 0, 'maintenanceDirectCents'),
    maintenance_reserve: intOr(input.maintenanceReserveCents, 0, 'maintenanceReserveCents'),
    airport_charges: intOr(input.airportChargesCents, 0, 'airportChargesCents'),
    navigation_charges: intOr(input.navigationCents, 0, 'navigationCents'),
    catering: intOr(input.cateringCents, 0, 'cateringCents'),
    handling: intOr(input.handlingCents, 0, 'handlingCents'),
    de_icing: intOr(input.deIcingCents, 0, 'deIcingCents'),
  };
};

```

shared/src/costing.js

Three patterns in this function are worth naming:

1. **Every optional field defaults explicitly.** `input.fuelKg ?? 0` means a missing fuel figure is treated as zero, not left undefined and multiplied later.
2. **A helper validates every monetary input the same way.** `intOr(x, 0, 'name')` throws if the value is present but not integer cents. This is how the float prohibition is enforced across every cost line.
3. **Overflow is caught before it silently corrupts.** The check on `fuelCents` exists because a single absurd fuel figure could exceed the safe range and lose digits.

The unknown-category rule

```

/** @type {Record<string, number>} */
const byCategory = { FLIGHT_ATTRIBUTABLE: 0, FLEET_FIXED: 0, PERIOD: 0, UNKNOWN: 0 };
for (const [category, amount] of Object.entries(lines)) {
  if (FLIGHT_ATTR.has(category)) byCategory.FLIGHT_ATTRIBUTABLE += amount;
  else if (FLEET_FIXED.has(category)) byCategory.FLEET_FIXED += amount;
  else if (PERIOD.has(category)) byCategory.PERIOD += amount;
  else byCategory.UNKNOWN += amount;
}

if (byCategory.UNKNOWN !== 0) {
  throw new MoneyError(
    `Cost build contains ${byCategory.UNKNOWN} cents of unrecognised cost categories`,
    'COST_UNKNOWN_CATEGORY',
  );
}

```

shared/src/costing.js — an unrecognised category is an error, not a silent drop.

This is the philosophy of the codebase in eight lines

The comment above it says: "An unrecognised category is an error, not a silent drop — silent drops in a costing engine become unreconciled variances that nobody can find months later." A cost that disappears without trace is far more dangerous than a system that refuses to run. **Failing loudly is a feature.**

Allocating fixed cost to flights

```

export const ALLOCATION_DRIVERS = [
  'BLOCK_HOURS',      // standard: cost that scales with flying time
  'ASKS',              // seats x km: scales with capacity offered
  'SECTORS',          // per take-off: scales with movements
  'DISTANCE_KM',      // scales with distance
  'FLIGHTS',          // flat per rotation
  'DIRECT',           // already flight-specific, no allocation
];

```

shared/src/domain.js — the permitted ways to share a fixed cost.

```

export function allocateFleetFixed({ poolCents, flights, driver = 'BLOCK_HOURS' }) {
  if (!ALLOCATION_DRIVERS.includes(driver)) {
    throw new MoneyError(`Unknown allocation driver "${driver}"`, 'COST_BAD_DRIVER');
  }
  if (!Number.isSafeInteger(poolCents)) {
    throw new MoneyError('poolCents must be integer cents', 'COST_BAD_POOL');
  }
  if (flights.length === 0) {
    if (poolCents !== 0) throw new MoneyError('Non-zero pool across zero flights', 'COST_NO_FLIGHTS');
    return { allocations: [], driver, totalCents: 0 };
  }

  if (driver === 'DIRECT') {
    // Caller already attributed these; nothing to spread.
    return { allocations: flights.map(() => 0), driver, totalCents: 0 };
  }

  const weights = flights.map((f) => {
    switch (driver) {
      case 'BLOCK_HOURS': return f.blockHours ?? 0;
      case 'ASKS': return (f.seatsOffered ?? 0) * f.distanceKm;
      case 'SECTORS': return f.sectors ?? 1;
      case 'DISTANCE_KM': return f.distanceKm;
      case 'FLIGHTS': return 1;
      default: throw new MoneyError(`Unhandled driver "${driver}"`, 'COST_BAD_DRIVER');
    }
  });

  const allocations = allocate(poolCents, weights);
  // Self-check. If this ever throws, a rounding rule was changed and
  // the whole costing model needs re-verification.
  assertReconciles(allocations, poolCents, `fleet-fixed allocation (${driver})`);
  return { allocations, driver, totalCents: sum(...allocations) };
}

```

shared/src/costing.js

A lease of US\$400,000 a month for one aircraft is a real cost that does not vary with how much you fly. But the aircraft *does* fly specific sectors, and those sectors should carry a share. The choice of driver is a management policy, so it is made explicit and recorded rather than buried.

The self-check on the last line

`assertReconciles(allocations, poolCents, ...)` verifies that the parts still sum to the whole *after* allocation. The comment says that if this ever throws, a rounding rule has changed and the costing model needs re-verification. The engine checks its own output on every call rather than trusting it.

Full profit and loss for a flight

```

export function flightPnl({ costInput, fleetFixedLines = [], revenueCents, otherRevenueCents = 0 }) {
  const direct = buildFlightCost(costInput);
  const fixedTotal = sum(...fleetFixedLines.map((l) => l.cost));

  const fullCost = direct.totalCents + fixedTotal;
  const totalRevenue = intOr(revenueCents, 0, 'revenueCents') + intOr(otherRevenueCents, 0, 'otherRevenueCents');
  const metrics = flightMetrics({
    seatsOffered: costInput.seatsOffered ?? 0,
    passengers: costInput.passengers ?? 0,
    distanceKm: costInput.distanceKm,
    revenueCents: totalRevenue,
    operatingCostCents: fullCost,
  });

  return {
    ...metrics,
    // Physical facts carried through. Without these the aggregation layer
    // cannot sum passengers, seats, distance or block hours, and any
    // passenger-level or utilisation metric computed above the flight is
    // fabricated rather than derived.
    seatsOffered: costInput.seatsOffered ?? 0,
    passengers: costInput.passengers ?? 0,
    distanceKm: costInput.distanceKm,
    blockHours: costInput.blockHours ?? 0,

    revenueCents: totalRevenue,
    directCostCents: direct.totalCents,
    fleetFixedCostCents: fixedTotal,
    fullCostCents: fullCost,
    contributionCents: totalRevenue - fullCost,

    marginalCostOfExtraSeatCents: estimateMarginalSeatCost(costInput),
    breakEvenRevenueCents: fullCost,

    lines: direct.lines,
    fleetFixedLines,
  };
}

```

shared/src/costing.js — the flight profit and loss statement.

The block comment on the physical facts is instructive. At one point this function returned only ratios and totals. The aggregation layer above it then had no way to sum passengers or distance, so any route-level utilisation figure was computed from zeros. That is Defect 1 in Part XIII, and the comment was added afterwards so the error cannot recur.

Marginal cost: what one more passenger costs

```

export function estimateMarginalSeatCost(input) {
  const PASSENGER_ALL_UP_KG = 100; // ~75 kg person + ~25 kg baggage
  const BURN_KG_PER_KG_PAYLOAD = 0.030; // ~3.0% of payload as incremental fuel
  const fuelPricePerKg = input.fuelPriceCentsPerKg ?? 0;
  const fuel = Math.round(PASSENGER_ALL_UP_KG * BURN_KG_PER_KG_PAYLOAD * fuelPricePerKg);
  // Onboard service scales with passengers served.
  const service = input.seatsOffered && input.seatsOffered > 0
    ? Math.round(((input.cateringCents ?? 0) + (input.handlingCents ?? 0)) / input.seatsOffered)
    : 0;
  return fuel + service;
}

```

shared/src/costing.js — an extra passenger weighs 100 kg all-up, and carrying weight costs roughly 3% of it in extra fuel. Catering and handling are divided by seats because they scale with load.

```

// At 95 cents per kg:
100 kg x 0.030 x 95c = 285 cents for fuel
Catering + handling per seat = ~700 cents
Marginal cost of an extra seat = 985 cents (US$9.85)

```

An honest limitation of this estimate

The 3% burn figure and the 100 kg all-up weight are industry approximations, not measurements from your aircraft. They are also named constants, visible at the top of the function, precisely so a reader can see they are assumptions rather than derived facts. A qualified engineer should validate them against your fleet's actual fuel data before the number drives a pricing decision.

Aggregating many flights

```

const t = flightPnls.reduce(
  (a, f) => ({
    flights: a.flights + 1,
    asks: a.asks + f.asks,
    rpks: a.rpks + f.rpks,
    revenueCents: a.revenueCents + f.revenueCents,
    directCostCents: a.directCostCents + f.directCostCents,
    fleetFixedCostCents: a.fleetFixedCostCents + f.fleetFixedCostCents,
    fullCostCents: a.fullCostCents + f.fullCostCents,
    seatsOffered: a.seatsOffered + (f.seatsOffered ?? 0),
    passengers: a.passengers + (f.passengers ?? 0),
    distanceKm: a.distanceKm + (f.distanceKm ?? 0),
    blockHoursMilli: a.blockHoursMilli + Math.round((f.blockHours ?? 0) * 1000),
    marginalSeatCostCents: a.marginalSeatCostCents + (f.marginalCostOfExtraSeatCents ?? 0),
  }),
  { flights: 0, asks: 0, rpks: 0, /* ... */ },
);

assertReconciles([t.directCostCents, t.fleetFixedCostCents], t.fullCostCents, 'aggregate flight cost');

```

shared/src/costing.js

Two details here are deliberate and were both added in response to real problems.

```

    blockHoursMilli: a.blockHoursMilli + Math.round((f.blockHours ?? 0) * 1000),
    // ...
    blockHours: t.blockHoursMilli / 1000,
    // ...
    marginalSeatCostCents: t.marginalSeatCostCents / t.flights,

```

- **Block hours are accumulated in thousandths.** Summing `Math.round(blockHours * 1000)` as integers and dividing once at the end avoids the drift described in Part IV.
- **Marginal seat cost is averaged across flights,** then used for the break-even passenger calculation. The comment says this is "carried because the break-even passenger gap is ONLY computable from the airline's real marginal cost, not from a constant."

The route verdict

```

let verdict;
if (agg.flights === 0) verdict = 'NO_DATA';
else if (agg.contributionCents < 0) verdict = 'LOSS_MAKING';
else if (agg.loadFactorPpm < targetLoadFactorPpm) verdict = 'MARGINAL';
else verdict = 'PROFITABLE';

return {
  verdict,
  structurallyImpossible,
  marginPpm,
  /** Exact arithmetic from real revenue-per-pax and real marginal cost. */
  breakEven: breakEvenPassengers(agg),
  levers: {
    requiredLoadFactorPpm: agg.breakEvenLoadFactorPpm,
    currentLoadFactorPpm: agg.loadFactorPpm,
    loadFactorGapPpm: agg.loadFactorPpm - agg.breakEvenLoadFactorPpm,
    requiredCaskMicrocents: agg.raskMicrocents,
    currentCaskMicrocents: agg.caskMicrocents,
    caskHeadroomMicrocents: agg.raskMicrocents - agg.caskMicrocents,
  },
};

```

shared/src/costing.js — what a commercial manager actually sees.

The `levers` object is the interesting part. Rather than returning "loss-making" and stopping, it separates two distinct problems: the route is under-performing commercially (load factor too low) or operationally (unit cost too high). Those need different responses, and a verdict that conflated them would send management in the wrong direction.

```

/**
 * Note on taxonomy. Because full cost per flight is modelled as
 * load-independent, "negative contribution" and "requires more than
 * 100% load factor to break even" are the same condition. An earlier
 * version exposed both as separate verdicts, which made one of them
 * unreachable. They are now separated cleanly: `verdict` describes the
 * money, and `structurallyImpossible` describes the achievability.
 */

```

What that note is really about

A verdict that can never be reached is a latent bug. The team found one branch was unreachable under the current cost model and said so in the code rather than leaving a condition that looks meaningful and never fires. This is unusually candid and it is the right instinct: the next engineer to change the cost model needs to know that this distinction is conditional on how the model treats load.

Part VI — Fuel, the largest single cost

Fuel is typically a quarter to a third of an airline's operating cost, and it is the one cost the airline does not control, cannot renegotiate, and consumes before it earns.

```
/**
 * AMS — Fuel management and burn variance.
 *
 * Fuel is typically 25-35% of an airline's operating cost and is the
 * single largest lever on unit economics. A 2% saving on fuel equals
 * roughly 60 basis points of CASK, which is a strategic margin in a
 * low-margin market. It is also the one cost an airline does not
 * control, cannot renegotiate, and consumes before it earns.
 *
 * Three things a finance system must get right about fuel:
 *
 * 1. UPLIFT IS NOT BURN. The aircraft leaves with uplifted litres and
 * lands with reserves (taxi, trip fuel, contingency, alternate,
 * final reserve, and ETOPS reserve where applicable). Costing the
 * whole uplift to the flight overstates the flight and understates
 * the next one. AMS reconciles uplift-to-burn explicitly and carries
 * the landing reserve forward.
 *
 * 2. PLANNED BURN IS NOT ACTUAL BURN. The crew files a plan derived
 * from route, weight, weather and winds. Actual burn comes from the
 * aircraft. The variance between them is the single most useful
 * operational-financial metric an airline produces, because it
 * isolates crew technique, routing efficiency and weight discipline
 * from the fuel price, which is pure market risk.
 *
 * 3. CARRYING FUEL HAS AN OPPORTUNITY COST. Tankering — deliberately
 * carrying extra fuel — trades deadweight (which burns fuel and
 * costs payload) against the risk of paying a diversion penalty or
 * buying fuel at an expensive alternate. That is a calculable
 * trade, and AMS calculates it rather than leaving it to habit.
 *
 * @module fuel
 */
```

shared/src/fuel.js

Uplift is not burn

An aircraft that takes on 12,000 kg does not use 12,000 kg. It lands with several hundred kilograms still on board — enough for the taxi, a contingency, an alternate airport it may need, and a mandatory final reserve.

```

export function reconcileFuel(plan, actual) {
  const reserves = plan.taxiFuelKg + plan.contingencyKg + plan.alternateKg
    + plan.finalReserveKg + (plan.etopsReserveKg ?? 0);

  const burnable = plan.taxiFuelKg + plan.tripFuelKg + plan.contingencyKg;
  const actualBurn = actual.takeoffFuelKg - actual.landingFuelKg;

  const planBurn = plan.taxiFuelKg + plan.tripFuelKg;
  const varianceKg = actualBurn - planBurn;
  const variancePpm = planBurn === 0 ? 0
    : Math.floor((varianceKg / planBurn) * 1_000_000 + 0.5);

  const expectedLanding = plan.upliftKg - burnable;

  return {
    reservesKg: reserves,
    reservesMeetRegulatoryMinimum: expectedLanding >= reserves,
    plannedBurnKg: planBurn,
    actualBurnKg: actualBurn,
    varianceKg,
    variancePpm,
    /** Positive variance is bad: more than planned was burned. */
    varianceDirection: varianceKg > 0 ? 'OVER' : varianceKg < 0 ? 'UNDER' : 'ON_PLAN',
    expectedLandingFuelKg: expectedLanding,
    actualLandingFuelKg: actual.landingFuelKg,
    landingReserveVarianceKg: actual.landingFuelKg - expectedLanding,
    costCents: Math.round(actualBurn * plan.priceCentsPerKg),
    /** Cost of the variance alone – this is what management can influence. */
    varianceCostCents: Math.round(varianceKg * plan.priceCentsPerKg),
  };
}

```

shared/src/fuel.js

The last two fields are the commercially valuable ones. `costCents` is what the flight cost in fuel. `varianceCostCents` is what the difference between plan and reality cost — which is the part management can actually influence. A 3% burn variance across a fleet is a seven-figure annual number, and it is invisible in a monthly profit and loss because it is buried inside the fuel line.

Why the reserve check compares plan to expectation, not to reality

`reservesMeetRegulatoryMinimum` compares the expected landing fuel against the required reserves. This is a *planning* check performed before dispatch. Whether the aircraft actually landed with enough is a separate fact, reported by `landingReserveVarianceKg`. Conflating the two would let a planning error be hidden by a lucky landing, or blame the crew for a planning error.

Reconciling plan against actual

```

// Apply empirically observed variance so the aircraft is never
// dispatched below regulatory reserve because the crew flew well
// last time.
const bufferPpm = ctx.expectedBurnVariancePpm ?? 30_000; // default +3%
const buffered = Math.ceil((requiredWithContingency * (1_000_000 + bufferPpm)) / 1_000_000);

```

shared/src/fuel.js — the uplift recommendation carries an empirical safety margin.

That comment describes a real accident pattern. An airline notices its crews consistently beat the fuel plan, and gradually trims the uplift. One day the flight meets headwinds and the aircraft lands below its legal final reserve. AMS refuses to learn from a favourable variance by reducing the required uplift.

Tankering: carrying fuel is a decision

```
// Tankering economics.
//
// The naive formulation – "carrying fuel is worth what fuel costs at
// the alternate" – makes tankering look free, because it ignores the
// two costs that actually matter:
//
// 1. Diversion only HAPPENS sometimes. The value of carrying
//    contingency fuel is the EXPECTED cost avoided, so it is scaled
//    by the probability of a diversion, not the gross cost of one.
// 2. Carrying fuel is not free. It burns on every remaining sector
//    of the rotation, and it may displace fuel you could otherwise
//    uplift cheaply at the next station.
//
// Getting this wrong in either direction is expensive and it is a
// decision made thousands of times a year, so AMS calculates it.
const deadweightBurnFactor = ctx.deadweightBurnFactor ?? 0.030; // kg fuel per kg payload
const probabilityOfDiversion = ctx.probabilityOfDiversion ?? 0.02;
const remainingSectors = ctx.remainingSectors ?? 3;

const costIfDivertedCents =
  ctx.costOfDiversionCents + ctx.fuelPriceAtAlternateCentsPerKg * plan.alternateKg;
const expectedSavingCents = Math.round(probabilityOfDiversion * costIfDivertedCents);

const carryingCostCents = Math.round(
  plan.contingencyKg * ctx.fuelPriceHereCentsPerKg
  * (1 + deadweightBurnFactor * remainingSectors),
);

const shouldTanker = expectedSavingCents > carryingCostCents;
```

shared/src/fuel.js

This is a well-constructed expected-value calculation, and it is worth understanding as a template because the reasoning generalises. The question "is it worth carrying this?" is answered by comparing the expected benefit against the expected cost — not the maximum benefit against the certain cost, which is how organisations routinely make defensible-looking but wrong capital decisions.

```
// Carrying 300 kg of contingency fuel:
//   Benefit if diverted: $4,200 diversion + alternate fuel
//   x 2% probability    = $84 expected saving
//   Cost: 300 kg x 95c x (1 + 0.03 x 3 sectors) = $31
//   Verdict: TANKER (net benefit $53)
```

The defaults here are assumptions, not findings

A 2% diversion probability and a 3% deadweight burn factor are industry-plausible but unverified. Both are named constants with comments, and both can be overridden per call. Before this drives real tankering decisions, your flight operations team should replace them with figures from your own data.

Statistical detection of a fuel problem

```
export function detectBurnVarianceAnomaly(variancePpmSeries, { window = 30, zThreshold = 2.5 } = {}) {
  if (variancePpmSeries.length < window) {
    return { status: 'INSUFFICIENT_DATA', sampleSize: variancePpmSeries.length };
  }

  const n = variancePpmSeries.length;
  const mean = variancePpmSeries.reduce((a, b) => a + b, 0) / n;
  const variance = variancePpmSeries.reduce((a, b) => a + (b - mean) ** 2, 0) / n;
  const stdev = Math.sqrt(variance);
  if (stdev === 0) {
    return { status: 'STABLE', meanPpm: Math.round(mean), stdev: 0, anomalies: [] };
  }

  /** @type {{index:number, value:number, zScore:number}[]} */
  const anomalies = [];
  variancePpmSeries.forEach((value, index) => {
    const z = (value - mean) / stdev;
    if (Math.abs(z) >= zThreshold) {
      anomalies.push({ index, value, zScore: Math.floor(z * 100) / 100 });
    }
  });

  return {
    status: anomalies.length === 0 ? 'STABLE' : 'ANOMALY_DETECTED',
    sampleSize: n,
    meanPpm: Math.round(mean),
    stdevMilli: Math.floor(stdev * 1000),
    anomalies,
    interpretation: mean < -20_000
      ? 'Fleet is consistently beating plan – verify distance data is not overstated'
      : mean > 40_000
      ? 'Fleet is consistently over plan – investigate routing, weight discipline, or uplift recording'
      : 'Burn performance within expected range',
  };
}
```

shared/src/fuel.js — z-score detection of a systematic fuel problem.

This is a standard statistical technique given an aviation-specific interpretation, and that interpretation is the valuable part. A consistent *negative* variance means crews are beating plan — which sounds good and is actually a warning sign that the planning distance data is overstated, which in turn means every route profitability figure built on it is wrong.

```
// `**2` is the exponent operator – "square this".
// Math.sqrt(variance) is the square root.
// The whole calculation is textbook: mean, variance, standard deviation,
// then measure how far each point sits from the mean in standard
// deviations.
```

Why this is in the finance system and not the flight ops system

Because the finding is a *financial* one. A systematic 4% burn overrun across a fleet is worth six figures annually, and it is invisible in a monthly profit and loss where it is averaged into the fuel line alongside genuine price movements. The statistic exists to surface a cost that would otherwise be absorbed silently.

Part VII — Airworthiness and legal safety

This is the part of AMS where a software error can cost an airline its operating certificate. The code is written with a different level of caution, and it should be reviewed with more scepticism than the rest of the system.

```
/**
 * AMS — Airworthiness: life-limited parts and AD/SB compliance.
 *
 * THIS IS WHERE AN AIRLINE LOSES ITS CERTIFICATE IF IT GETS IT WRONG.
 *
 * A life-limited part (LLP) — an engine C-rib, a main landing gear,
 * a flight control component — has a hard life limit measured in
 * flight cycles, flight hours, or calendar months, set by its certifying
 * authority and never extended. Exceed it and the part must be
 * destroyed. Operate with an LLP beyond limit and the operator's Air
 * Operator Certificate is at risk, aircraft are grounded, and the
 * liability is personal as well as corporate.
 *
 * An Airworthiness Directive (AD) is a mandatory instruction issued by a
 * regulator, often after a fleet-wide safety finding. A Service Bulletin
 * (SB) is the manufacturer's version — often technically equivalent but
 * not, by itself, legally mandatory until adopted by an AD. Conflating
 * them is a common and expensive error: a fleet that "applied the SB" is
 * not necessarily compliant with the AD.
 *
 * AMS therefore tracks ADs and SBs as SEPARATE compliance tracks. An
 * SB applied does not close an AD. Only the AD does.
 *
 * @module airworthiness
 */
```

shared/src/airworthiness.js — note the opening line. The author is stating the stakes before writing any code.

Life-limited parts and three counters

```
export function llpStatus(llp) {
  const cycles = remaining(llp.cyclesTotalLimit, llp.cyclesSinceNew);
  const hours = remaining(llp.hoursTotalLimit, llp.hoursSinceNew);
  const months = remaining(llp.monthsTotalLimit, llp.monthsSinceNew);

  /** @type {{counter: string, remaining: number | null, unit: string}[]} */
  const counters = [];
  if (cycles !== null) counters.push({ counter: 'CYCLES', remaining: cycles, unit: 'C' });
  if (hours !== null) counters.push({ counter: 'HOURS', remaining: hours, unit: 'H' });
  if (months !== null) counters.push({ counter: 'CALENDAR', remaining: months, unit: 'M' });

  if (counters.length === 0) {
    throw new TypeError('LLP must declare at least one life limit');
  }
}
```

A life-limited part wears out on three independent clocks at once. Cycle life counts landings and takeoffs. Hour life counts flying time. Calendar life counts months since the part was new. A part must be removed when *any* clock runs out.

Counter	What it measures	Typical limit
Cycles	Takeoffs and landings	20,000–45,000
Hours	Actual flying time	40,000–80,000
Calendar	Months since new, regardless of use	120–192 months

The example in the AMS README makes the point exactly

A part with 50% of its cycle life left but 4% of its calendar life remaining is nearly scrap. A planner who looks only at cycles will fly it into the ground. This is not a subtle edge case — it is one of the most common real errors in life-limited part planning.

Finding the limit that binds first

```
// Controlling limit = least remaining life, normalised to a percentage
// of each counter's own total so the comparison is meaningful.
const normalised = counters
  .map((c) => {
    const total =
      c.counter === 'CYCLES' ? llp.cyclesTotalLimit
      : c.counter === 'HOURS' ? llp.hoursTotalLimit
      : llp.monthsTotalLimit;
    return { ...c, remainingPpm: Math.floor((c.remaining / total) * 1_000_000) };
  })
  .sort((a, b) => a.remainingPpm - b.remainingPpm);

const controlling = normalised[0];
const status = statusFromRemainingPpm(controlling.remainingPpm);

return {
  partNumber: llp.partNumber,
  serialNumber: llp.serialNumber,
  counters,
  controllingCounter: controlling.counter,
  controllingRemaining: controlling.remaining,
  controllingRemainingPpm: controlling.remainingPpm,
  status,
  mustRemove: status === 'OVERDUE',
  /** An LLP inside the warning band is a planning item, not a surprise. */
  planReplacement: status !== 'COMPLIANT',
};
```

You cannot compare "5 cycles remaining" against "5 months remaining" directly. The code therefore normalises each counter to a percentage of its own limit, then takes the smallest. This is where Part III's technique of storing rates as integers appears again.

```
llpStatus({ partNumber: 'P/N-4471', cyclesTotalLimit: 10000, cyclesSinceNew: 5000,
  monthsTotalLimit: 120, monthsSinceNew: 115 });
// → { controllingCounter: 'CALENDAR', controllingRemaining: 5, mustRemove: false }
```

Directives and bulletins are not the same

```
export const ATA_CHAPTERS = [
  '21', '22', '23', '24', '25', '26', '27', '28', '29', '30', '31', '32',
  '33', '34', '35', '36', '38', '45', '46', '47', '48', '49', '51', '52',
  '53', '54', '55', '56', '77',
];

/**
 * ATA chapter codes.
 *
 * Airworthiness Directives are published under an ATA chapter, and
 * technical and compliance staff search by ATA chapter – "what is
 * outstanding on landing gear?" – not by AD reference number. Without
 * this field every such query becomes a full-text scan.
 *
 * Source: structure observed directly in 14 CFR Part 39 ADs published
 * in the Federal Register (79 FR 19848; 87 FR 67354) and in live EASA
 * directives AD 2026-0125-E, AD 2023-0167R1 and AD 2024-0213.
 */
```

shared/src/domain.js — and note that the source citations are recorded in the code.

What to notice about that comment

The data structure was derived from documents the author claims to have examined directly, and the citations are specific: two Federal Register page references and three named EASA directives. That is a different standard from asserting a regulatory fact without a source, and it is what allows a reviewer to check the claim rather than take it on trust.

```
export function adsbApplies(ad, aircraft) {
  if (ad.superseded) {
    return { applies: false, reason: 'Superseded by a later directive', complianceStatus: null };
  }
  if (ad.aircraftType !== aircraft.aircraftType) {
    return { applies: false, reason: `Not applicable to ${aircraft.aircraftType}`, complianceStatus: null };
  }
  const [from, to] = ad.serialRange;
  if (aircraft.serialNumber < from || aircraft.serialNumber > to) {
    return {
      applies: false,
      reason: `Serial ${aircraft.serialNumber} outside affected range ${from}-${to}`,
      complianceStatus: null,
    };
  }
  // ... thresholds, controlling counter, status ...
}
```

shared/src/airworthiness.js — applicability is decided before any threshold arithmetic.

Real applicability is not a single test. An airworthiness directive names a serial range, applies to a specific aircraft type, and may be superseded by a later directive. All three must be checked, and each returns a *reason* so an engineer can see why it was excluded rather than just seeing a blank.

```

/**
 * An SB applied does NOT satisfy an AD. This flag is the reason
 * AMS keeps the two tracks separate.
 */
requiresMandatoryInstruction: ad.kind === 'AD',

```

Deferral deadlines and the off-by-one

```

export const MEL_CATEGORIES = {
  A: { days: null, note: 'No standard interval specified; starts at deferral' },
  B: { days: 3, note: '3 calendar days excluding the day of discovery' },
  C: { days: 10, note: '10 calendar days excluding the day of discovery' },
  D: { days: 120, note: '120 calendar days excluding the day of discovery' },
};

/**
 * Source: EASA CS-GEN-MMEL Issue 2, Annex II to ED Decision 2020/012/R.
 */

```

```

export function rectifyDeadline(category, discoveredAtMs) {
  const spec = MEL_CATEGORIES[category];
  if (!spec) throw new TypeError(`Unknown MEL category "${category}"`);
  if (spec.days === null) return null; // Category A: no standard interval

  const discovery = new Date(discoveredAtMs);
  const due = new Date(discovery.getTime());
  // The day of discovery is excluded, so counting starts at day 1 = the
  // following calendar day.
  due.setUTCDate(due.getUTCDate() + spec.days);
  due.setUTCHours(23, 59, 59, 999); // valid to the END of that day
  return due.getTime();
}

```

This function is a worked example of why domain detail matters more than arithmetic elegance. Three separate things are being got right.

1. **The day of discovery does not count.** A Category B defect found on Monday is not due on Thursday — it is due on the *end* of Thursday.
2. **It runs to the end of the day, not the same time of day.** `setUTCHours(23, 59, 59, 999)` is what makes it 23:59 on Thursday rather than 09:00.
3. **Everything is UTC.** `setUTCDate`, not `setDate`. The operator's local timezone must not shift a legal deadline.

```

// Category B defect discovered 09:00 Monday 6 July 2026
rectifyDeadline('B', ms) // → end of Thursday 9 July 2026

// A flight operating Friday 10 July must be dispatched as EXPIRED.

```

Why this specific off-by-one is dangerous

Getting it wrong in the *lenient* direction means an aircraft flies with an expired deferral. Getting it wrong in the strict direction grounds a perfectly airworthy aircraft unnecessarily. Both are expensive; only one is a safety breach. The code comments state the source document so the arithmetic can be re-checked against the regulation by someone qualified to read it.

Why a deferral cannot cancel a directive

```

/**
 * Aggregate every compliance item on one aircraft into a single
 * airworthiness gate, enforcing the subordination of the MEL.
 *
 * CRITICAL: EASA CS-GEN-MMEL states plainly that
 * "The MEL cannot deviate from Airworthiness Directives or any other
 * additional mandatory requirements."
 *
 * A MEL deferral is therefore legally INCAPABLE of curing an
 * outstanding AD. This function never marks an AD as deferred, and it
 * separately reports any attempt to do so – because silently ignoring
 * an illegal deferral would let an operator believe an aircraft is
 * airworthy when it is not.
 *
 * @param {ADSB[]} registry
 * @param {{ aircraftType: string, serialNumber: number, cycles: number, hours: number, nowMs: number }} aircraft
 * @param {Array<{ itemRef: string }>} [melDeferrals]
 */
export function airworthinessGate(registry, aircraft, melDeferrals = []) {
  const applicable = registry
    .map((ad) => adsbApplies(ad, aircraft))
    .filter((r) => r.applies);

  const deferredRefs = new Set(melDeferrals.map((d) => d.itemRef));

  const overdue = [];
  const dueSoon = [];
  /** @type {string[]} */
  const illegalDeferralAttempts = [];

  for (const r of applicable) {
    const cs = r.complianceStatus;
    // An AD may never be recorded as deferred, whatever the operator
    // believes. Only a matching Service Bulletin is deferrable.
    const attempt = deferredRefs.has(cs.reference);
    cs.deferred = false;
    cs.deferralAttempted = attempt;

    if (cs.status === 'OVERDUE') {
      overdue.push(cs);
      if (attempt) illegalDeferralAttempts.push(cs.reference);
    } else if (cs.status === 'DUE_SOON') {
      dueSoon.push(cs);
    }
  }

  return {
    dispatchable: overdue.length === 0,
    applicableCount: applicable.length,
    overdueCount: overdue.length,
    dueSoonCount: dueSoon.length,
    overdue,
    dueSoon,
    illegalDeferralAttempts,
    nextDue: [...overdue, ...dueSoon].sort((a, b) => a.remainingPpm - b.remainingPpm)[0] ?? null,
  };
}

```

This is the most important design decision in the codebase

Note that `cs.deferred = false` is assigned *unconditionally*. The system does not merely refuse to mark an overdue directive as deferred — it sets the flag to false regardless of what the operator requested. And when someone *has* attempted it, the reference is collected into `illegalDeferralAttempts` rather than being silently dropped.

There is a meaningful difference here between a system that blocks an illegal action and a system that quietly ignores one. If the engineering team records a deferral against an overdue airworthiness directive, that is a signal that someone believes the aircraft is airworthy when it is not — or that someone is trying to record something that is not permitted. Both warrant investigation. Silently discarding the attempt would hide the first and enable the second.

The dispatch gate

```
export function checkDue({ intervalHours, intervalCycles, hoursAtLastCheck,
  cyclesAtLastCheck, currentHours, currentCycles, utilisationHoursPerDay = 10 }) {
  const hoursUsed = currentHours - hoursAtLastCheck;
  const cyclesUsed = currentCycles - cyclesAtLastCheck;

  const hoursRemaining = intervalHours - hoursUsed;
  const cyclesRemaining = intervalCycles - cyclesUsed;

  // Which runs out first, expressed as calendar days at current rate?
  const daysToHoursLimit = utilisationHoursPerDay > 0
    ? Math.floor(hoursRemaining / utilisationHoursPerDay) : Number.MAX_SAFE_INTEGER;
  // ~1.4 average sectors per block hour on a narrowbody mainline.
  const sectorsPerHour = 1.4;
  const daysToCyclesLimit = cyclesRemaining > 0
    ? Math.floor(cyclesRemaining / (utilisationHoursPerDay * sectorsPerHour)) : Number.MAX_SAFE_INTEGER;

  const controlling = daysToHoursLimit <= daysToCyclesLimit ? 'HOURS' : 'CYCLES';
  const daysRemaining = Math.max(0, Math.min(daysToHoursLimit, daysToCyclesLimit));

  return {
    hoursRemaining, cyclesRemaining, daysRemaining, controllingCounter: controlling,
    status: daysRemaining <= 3 ? 'OVERDUE_OR_IMMINENT' : daysRemaining <= 21 ? 'DUE_SOON' : 'SCHEDULED',
  };
}
```

shared/src/airworthiness.js — converting two running counters into a planning date.

The step worth understanding is the conversion from "hours remaining" to "days remaining". A check due in 90 block hours sounds distant; at 10 hours a day it is 9 days away, which is a maintenance planning item right now. The comment attributes the 1.4 sectors-per-block-hour figure to narrowbody mainline operations — an assumption a reader can therefore challenge.

Part VIII — Routing and fuel planning (the algorithms)

This part covers the newest module in AMS, and the one with the strongest verification. It is also the longest, because route planning under a fuel constraint is genuinely hard.

The problem, stated precisely

An airline must decide which intermediate stops to use on a route. Every direct sector has a cost (crew, handling, overflight fees) and a fuel requirement. The aircraft can only carry a fixed amount of fuel, and must land with at least a mandatory reserve.

```
export function solveRoute({ flights, source, destination, tankCapacityKg, minReserveKg }) {
  if (!Number.isSafeInteger(tankCapacityKg) || tankCapacityKg < 0) {
    throw new RoutingError('tankCapacityKg must be a non-negative integer', 'ROUTE_BAD_CAPACITY');
  }
  if (!Number.isSafeInteger(minReserveKg) || minReserveKg < 0) {
    throw new RoutingError('minReserveKg must be a non-negative integer', 'ROUTE_BAD_RESERVE');
  }

  const usableFuelKg = tankCapacityKg - minReserveKg;
  if (usableFuelKg < 0) {
    throw new RoutingError(
      `Tank capacity ${tankCapacityKg}kg cannot hold the ${minReserveKg}kg mandatory reserve. ` +
      'The aircraft can never complete any sector.',
      'ROUTE_CAPACITY_BELOW_RESERVE',
    );
  }
}
```

In one sentence: **find the cheapest sequence of stops such that the fuel burned so far never exceeds capacity minus reserve.**

```
// A concrete case the planner must get right
JFK → YUL    38,000c    3,200 kg
YUL → GRU    74,000c    8,600 kg
JFK → GRU    61,000c   12,100 kg
JFK → DUB    22,000c    3,300 kg
DUB → GRU    89,000c   10,400 kg

Tank 24,000 kg, reserve 1,500 kg → 22,500 kg usable.
```

Route	Cost	Fuel	Fits in 22,500 kg?
JFK → GRU (direct)	61,000c	12,100 kg	Yes
JFK → YUL → GRU	112,000c	11,800 kg	Yes
JFK → DUB → GRU	111,000c	13,700 kg	Yes

With that tank, the direct route wins on cost. Now tighten the tank to 13,300 kg, giving 11,800 kg usable. The direct sector no longer fits, and the planner must take the *more expensive* two-stop route because the

cheap one cannot be flown.

```
const tight = solveRoute({ ...NETWORK, source: 'JFK', destination: 'GRU',
  tankCapacityKg: 13_300, minReserveKg: 1_500 });
tight.costCents // 112000 – more expensive, but it is the only legal answer
tight.stops     // ['JFK', 'YUL', 'GRU']

const impossible = solveRoute({ ...NETWORK, source: 'JFK', destination: 'GRU',
  tankCapacityKg: 13_299, minReserveKg: 1_500 });
impossible.feasible // false – one kilogram less and nothing is legal
```

The naive approaches and why they fail

```
// Approach 1: find the cheapest route, ignore fuel
const unconstrained = unconstrainedCheapest(inst); // 61,000c
// WRONG when the tank is tight. The aircraft cannot fly it.

// Approach 2: greedy – always take the cheapest next hop
// ALSO WRONG, and dangerously so. This counterexample is a test case
const inst = {
  flights: [
    { from: 'A', to: 'B', costCents: 10, fuelKg: 90 },
    { from: 'B', to: 'D', costCents: 10, fuelKg: 90 },
    { from: 'A', to: 'C', costCents: 11, fuelKg: 50 },
    { from: 'C', to: 'D', costCents: 11, fuelKg: 50 },
  ],
  source: 'A', destination: 'D',
  tankCapacityKg: 100, minReserveKg: 0,
};
const result = solveRoute(inst);
result.costCents // 22
result.stops     // ['A', 'C', 'D']
// Greedy picks A-B-D at cost 20 – but that burns 180kg in a 100kg tank.
// A planner that returns that route grounds the aircraft.
```

Why the greedy failure is so instructive

The greedy answer is *cheaper*. It is also impossible to fly. A planner that reported the greedy result would look better on every metric and would be catastrophically wrong. This is the shape of bug that no amount of dashboard review catches — the number looks right, because the question was never asked.

Keeping only useful possibilities

```
/**
 * DOMINANCE
 *
 * The algorithm keeps a SET of non-dominated labels at each airport.
 * A label is a pair (cost, fuelUsed) meaning "I can reach this airport
 * having spent this much money and this much fuel".
 *
 * Label L1 dominates L2 if:
 *     cost1 <= cost2  AND  fuel1 <= fuel2
 * and they are not identical.
 *
 * Intuition: if L1 is cheaper AND uses no more fuel, then L2 can never
 * lead to a better answer. Wherever L2 could go from here, L1 can go
 * too, more cheaply, and with at least as much fuel in the tanks.
 */
```

```

while (heap.size > 0) {
  const label = /** @type {Label} */ (heap.pop());
  labelsExpanded += 1;

  // Skip if this label was pruned after being queued.
  const live = labelsAt(label.node);
  if (!live.includes(label)) { labelsPruned += 1; continue; }

  if (label.node === destination) {
    // The heap is ordered by cost, so the first label popped at the
    // destination is the cheapest surviving one – and by the theorem,
    // the optimum.
    return {
      feasible: true,
      costCents: label.costCents,
      fuelUsedKg: label.fuelUsedKg,
      stops: reconstructStops(label),
      labelsExpanded, labelsPruned, maxFrontier,
    };
  }

  const outgoing = adjacency.get(label.node) ?? [];
  for (const edge of outgoing) {
    const fuelUsedKg = label.fuelUsedKg + edge.fuelKg;
    if (fuelUsedKg > usableFuelKg) continue; // infeasible prefix – never expand

    const costCents = label.costCents + edge.costCents;
    const target = labelsAt(edge.to);

    let dominated = false;
    for (const other of target) {
      if (dominates(other.costCents, other.fuelUsedKg, costCents, fuelUsedKg)) {
        dominated = true; break;
      }
      if (other.costCents === costCents && other.fuelUsedKg === fuelUsedKg) {
        dominated = true; break;
      }
    }
    if (dominated) { labelsPruned += 1; continue; }

    // Is it dominating something already there? Remove those.
    for (let i = target.length - 1; i >= 0; i -= 1) {
      if (dominates(costCents, fuelUsedKg, target[i].costCents, target[i].fuelUsedKg)) {
        target.splice(i, 1);
      }
    }

    const next = { costCents, fuelUsedKg, node: edge.to, prev: label };
    target.push(next);
    if (target.length > maxFrontier) maxFrontier = target.length;
    heap.push(next);
  }
}

```

shared/src/routing/constrained-path.js — the whole search loop.

The data structures

```

class MinHeap {
  constructor() { /** @type {Array<{label: Label, seq: number}>} */ (this.a = []); this.seq = 0; }

  /** @param {Label} label */
  push(label) {
    const a = /** @type {any} */ (this.a);
    a.push({ label, seq: this.seq++ });
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (a[p].label.costCents <= a[i].label.costCents) break;
      [a[p], a[i]] = [a[i], a[p]];
      i = p;
    }
  }

  pop() {
    const a = /** @type {any} */ (this.a);
    if (a.length === 0) return null;
    const top = a[0];
    const last = a.pop();
    if (a.length > 0) {
      a[0] = last;
      let i = 0;
      for (;;) {
        const l = 2 * i + 1, r = l + 1;
        let s = i;
        if (l < a.length && a[l].label.costCents < a[s].label.costCents) s = l;
        if (r < a.length && a[r].label.costCents < a[s].label.costCents) s = r;
        if (s === i) break;
        [a[s], a[i]] = [a[i], a[s]];
        i = s;
      }
    }
    return top.label;
  }

  get size() { return /** @type {any} */ (this).a.length; }
}

```

A priority queue written from scratch rather than imported — twenty lines, no dependency, and it is the only structure the algorithm needs.

A "heap" is a list arranged so the cheapest item is always at the top. The two arithmetic details worth recognising: $(i - 1) \gg 1$ finds the parent of position i in a binary tree stored in an array, and $2 * i + 1$ finds its left child. That is the entire trick.

The dominance rule and why it is sound

```

/**
 * Invariant I1 (reachability): every label in the frontier at v
 * corresponds to an actual feasible path from the source to v.
 *
 * Invariant I2 (pruning soundness): if L1 dominates L2 at v, then for
 * every feasible completion C of L2 to any destination, C composed
 * with L1 is also feasible and no more expensive.
 *
 * Theorem: the algorithm returns the minimum-cost feasible path.
 * Proof: Take an optimal feasible path P. Walk it from the source.
 * Each prefix label is either in the frontier, or was pruned. If
 * pruned, it was dominated by some retained L' which by I2 can replace
 * that prefix with no increase in cost or fuel. Following I2
 * inductively, a path of cost  $\leq \text{OPT}(P)$  survives to the destination.
 * By I1 every surviving label is feasible, so its cost  $\geq \text{OPT}(P)$ .
 * Therefore the minimum-cost surviving label has cost exactly  $\text{OPT}(P)$ .
 */

```

Translated out of notation: because both the objective (cost) and the constraint (fuel) improve in the same direction, a worse label can never rescue a better one. That is what makes discarding labels safe, and it is the entire reason the algorithm is exact rather than a heuristic.

Why this is worth reading even if you never write it

This is the standard of proof the module claims for itself, and it is written down rather than left implicit. When a qualified engineer reviews this code, this comment is what they check first: does the argument hold, and does the code match the argument? That question can only be asked if the argument was ever written down.

A second solver to check the first

```

/**
 * EXACT reference solver by fuel-indexed dynamic programming.
 *
 * Independent of the Pareto algorithm, so agreement between the two is
 * strong evidence of correctness. DP requires integer fuel values and
 * bounded capacity, which is fine for verification and small networks
 * but not for a whole airline's route network – hence the two exist.
 */

```

```

/** @type {Map<string, Map<number, {costCents: number, prevNode: string | null, prevFuel: number | null}>>}} */
const best = new Map();

/** @type {Array<Array<{node: string, ...}> | undefined>} */
const buckets = [];
buckets[0] = [{ node: source, costCents: 0, prevNode: null, prevFuel: null }];

for (let f = 0; f <= usable; f += 1) {
  const bucket = buckets[f];
  if (!bucket) continue;
  for (const state of bucket) {
    for (const edge of adjacency.get(state.node) ?? []) {
      const nf = f + edge.fuelKg;
      if (nf > usable) continue;
      const nc = state.costCents + edge.costCents;
      // ...
    }
  }
}

```

The idea is simpler: work through every possible amount of fuel that could have been burned, from zero upwards. Because fuel never decreases along a route, this is a valid order to process things in. For each fuel level, record the cheapest way to reach each airport.

The bug this solver had, and why two solvers are worth having

The DP originally recorded, for each state, the airport it had *departed* from rather than the one it had *arrived* at. Every subsequent step therefore re-expanded the wrong airport's outgoing flights. The consequence was that the DP reported routes as infeasible that the primary solver had found — and nobody noticed, because nothing compared the two.

This is the strongest argument in this guide for how AMS tests algorithms, and it is worth stating plainly: **a second implementation is only useful if something forces them to agree**. Two solvers that disagree and are never compared provide no more assurance than one.

Brute force as an oracle

```

/**
 * Exhaustive enumerator over every simple path. Exponential, used ONLY
 * to validate the fast algorithms on small instances.
 */

```

```

export function bruteForceRoute({ flights, source, destination, tankCapacityKg, minReserveKg }) {
  const adjacency = buildAdjacency(flights);
  let bestCost = Infinity;
  let bestStops = /** @type {string[] | null} */ (null);
  let explored = 0;

  const walk = (node, fuel, cost, path) => {
    explored += 1;
    if (cost >= bestCost) return; // cannot improve
    if (fuel > usable) return; // already infeasible
    if (node === destination) {
      bestCost = cost; bestStops = [...path, node]; return;
    }
    for (const e of adjacency.get(node) ?? []) {
      if (path.includes(e.to)) continue; // simple paths only
      walk(e.to, fuel + e.fuelKg, cost + e.costCents, [...path, node]);
    }
  };

  walk(source, 0, 0, [source]);
  return { feasible: bestStops !== null, costCents: bestCost === Infinity ? 0 : bestCost,
    stops: bestStops, pathsExplored: explored };
}

```

This solver tries every possible route, which is astronomically expensive on a real network and completely reliable on a small one. It is correct by construction — there is no cleverness in it to be wrong. That makes it an *oracle*: if the fast algorithm agrees with brute force on thousands of random networks, and brute force is obviously right, the fast algorithm is right.

```

describe('constrained shortest path – three independent solvers agree', () => {
  it('agrees with the exhaustive oracle on 3000 random instances', () => {
    let compared = 0;
    let feasible = 0;
    for (let seed = 1; seed <= 3000; seed += 1) {
      const rand = mulberry32(seed * 2654435761);
      const inst = randomInstance(rand, int(rand, 2, 6));
      if (inst.flights.length === 0) continue;

      const pareto = solveRoute(inst);
      const oracle = bruteForceRoute(inst);

      compared += 1;
      if (oracle.feasible) feasible += 1;

      expect(pareto.feasible, `seed ${seed}: feasibility must match oracle`).toBe(oracle.feasible);
      if (oracle.feasible) {
        expect(pareto.costCents, `seed ${seed}: optimal cost must match oracle`).toBe(oracle.costCents);
        const { cost } = assertRouteLegal(inst, pareto.stops);
        expect(cost, `seed ${seed}: reported cost must equal the sum of its own legs`).toBe(pareto.costCents);
      }
    }
    expect(compared).toBeGreaterThan(2500);
    // The test is worthless if every instance was trivially infeasible.
    expect(feasible / compared).toBeGreaterThan(0.3);
  });
});

```

tests/routing.test.js — the test that would have caught the DP bug and, in its current form, catches much else.

Making random tests reproducible

```
function mulberry32(seed) {
  let a = seed >>> 0;
  return function rand() {
    a |= 0; a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

tests/routing.test.js — a small pseudo-random number generator so that "seed 3" always produces the same test case.

Why reproducibility is a requirement

Without a seeded generator, a random test failure is unreproducible and therefore nearly useless. With one, "seed 3 fails" is a permanent, shareable, permanent regression case. Every random test in AMS names the seed in its failure message.

Guarding against a vacuous test

```
    compared += 1;
    if (oracle.feasible) feasible += 1;
    // ...
  }
  expect(compared).toBeGreaterThan(2500);
  // The test is worthless if every instance was trivially infeasible.
  expect(feasible / compared).toBeGreaterThan(0.3);
```

That last assertion is subtle and important. If a change to the instance generator produced only infeasible networks, every comparison would trivially pass — because both solvers would correctly report "no route" — and the test would prove nothing while reporting green. The assertion forces the test to confirm it is actually exercising interesting cases.

```
describe('the fuel constraint actually binds', () => {
  it('finds many instances where the unconstrained shortest path is INFEASIBLE', () => {
    let disagreements = 0;
    for (let seed = 1; seed <= 2000; seed += 1) {
      // ...
      if (constrained.feasible && constrained.costCents > unconstrained) disagreements += 1;
    }
    // A naive Dijkstra would be wrong this often. Proof the constraint matters.
    expect(disagreements).toBeGreaterThan(50);
  });
});
```

tests/routing.test.js — a test whose job is to prove the other tests are meaningful.

This is the shape of a mature test suite

It includes a test that verifies its own test data is interesting. Most suites omit this and can silently become decoration. The comment in the file says exactly that: if these ever stop finding instances, the suite has silently become vacuous, and a regression that dropped the fuel constraint would pass every other test in the file.

Optimal uplift planning

```
/**
 * THE THEOREM
 *
 * For a fixed route with LINEAR price per kg, capacity T and reserve R
 * required at every landing, the cost-optimal uplift policy is:
 *
 *   At stop i, let k = the first stop after i whose price is strictly
 *   cheaper than the price at i.
 *
 *   - If such a k exists: uplift only enough to reach k with the
 *   reserve intact. Buying more is waste, because  $p_k < p_i$ .
 *   Buying less is strictly more expensive.
 *
 *   - If no such k exists (every remaining stop is dearer, or we can
 *   reach the destination in one tank): buy as much as is still
 *   needed, filling the tank only if the remainder of the route
 *   does not fit.
 *
 * PROOF BY EXCHANGE
 * ...
 * This is verified independently against exhaustive search in
 * the test suite.
 */
```

shared/src/routing/refuel.js

A worked example. Route JFK → YUL → GRU, with MAN-style prices:

```
// JFK sells at 95c/kg, MAN at 70c/kg, and GRU is the destination.
const plan = optimalRefuelPlan({
  stops: ['JFK', 'MAN', 'GRU'],
  legFuelKg: [3200, 8600],
  priceCentsPerKg: [95, 70, 88],
  tankCapacityKg: 24_000,
  reserveKg: 1_500,
});
plan.upliftKg // [4700, 8600]
plan.totalCostCents // 4700*95 + 8600*70 = 1_048_500
```

Stop	Price	Arriving with	Buy	Why
JFK	95c	0 kg	4,700 kg	MAN at 70c is cheaper — buy only enough to reach it (3,200 leg + 1,500 reserve)
MAN	70c	1,500 kg	8,600 kg	Nothing cheaper ahead — buy what the rest of the route needs

Stop	Price	Arriving with	Buy	Why
GRU	—	1,500 kg	—	Arrival

If JFK had loaded all 13,300 kg for the whole journey, the cost would be 1,263,500 cents instead of 1,048,500 — 17% more, for no benefit. The planner correctly refuses to tanker past the cheap station.

The destination price trap

```
// GRU has the LOWEST price of the three nodes – but you cannot buy
// fuel on arrival, so its price must not influence any decision.
const plan = optimalRefuelPlan({
  stops: ['JFK', 'YUL', 'GRU'],
  legFuelKg: [3200, 8600],
  priceCentsPerKg: [95, 110, 88],
  tankCapacityKg: 24_000,
  reserveKg: 1_500,
});
plan.upliftsKg // [13_300, 0] – all loaded at JFK, the cheaper station
```

A test that encodes a subtle modelling decision

The destination price is in the array, is the lowest value, and is deliberately ignored. The code comment says why: "the destination entry is ignored". A test originally asserted the planner would stop at GRU because it looked cheap — which was wrong, and the test was the thing that had to change, not the code. This is a small example of a recurring theme: a plausible-looking expectation is not the same thing as a correct one.

Proving the uplift policy is optimal

```

* PROOF BY EXCHANGE
* -----
* Fix a stop i and suppose k = next cheaper stop exists.
*
* Under-buying: if the policy departs i with less fuel than needed to
* reach k, the shortfall must be bought at some stop j with  $i < j \leq k$ 
* (feasibility forbids running dry). Every such j has  $p_j \geq p_i$ , since
* k is the FIRST stop cheaper than i. Moving that purchase from j back
* to i cannot raise cost. It also cannot breach capacity: the shifted
* amount is exactly the shortfall, and the aircraft was already
* carrying more than the shortfall at i by hypothesis. Hence
* under-buying is never optimal.
*
* Over-buying: if the policy carries past i more fuel than needed to
* reach k, that surplus is consumed at a stop with  $p_k < p_i$ .
* Buying it at k instead is strictly cheaper. Capacity at i is not
* violated because we are removing, not adding. Hence over-buying is
* never optimal.
*
* No cheaper stop ahead: then  $p_j \geq p_i$  for all  $j > i$ . Every kilogram
* bought later can be bought at i for no greater price, subject to
* capacity. So buy the whole remaining requirement now; fill the tank
* only when the remainder does not fit.
*
* The three cases exhaust the possibilities, so the myopic policy is
* optimal.

```

Read plainly: if you buy too little at a stop, you will have to buy the shortfall somewhere more expensive later, so you should have bought it here. If you buy too much, you are carrying fuel that will be consumed somewhere cheaper, so you should have bought it there. The only defensible quantity is the minimum needed to reach the next cheaper station. There is nothing subtle here, which is why it is worth understanding.

```

describe('refuel - myopic policy versus exhaustive search', () => {
  it('matches the oracle on 2000 random routes', () => {
    let compared = 0;
    for (let seed = 1; seed <= 2000; seed += 1) {
      const rand = mulberry32(seed * 48271 + 31);
      const legs = int(rand, 1, 3);
      const args = {
        stops: Array.from({ length: legs + 1 }, (_, i) => `S${i}`),
        legFuelKg: Array.from({ length: legs }, () => int(rand, 1, 9)),
        priceCentsPerKg: Array.from({ length: legs + 1 }, () => int(rand, 1, 4) * 25),
        reserveKg: int(rand, 0, 2),
        tankCapacityKg: /* ... */ ,
      };
      const mine = optimalRefuelPlan(args);
      const oracle = bruteForceRefuel(args);
      expect(mine.totalCostCents).toBe(oracle.totalCostCents);
    }
  });
});

```

tests/routing.test.js — 2,000 random routes, each checked against exhaustive search over every possible uplift combination.

The five real bugs these tests caught

```
// The infinite loop, found because the test suite hung:
//
// The duplicate check is not cosmetic. A zero-cost, zero-fuel cycle
// produces a label identical to one already present. Identical
// labels do not strictly dominate one another, so without this the
// planner re-queues the same (cost, fuel) pair forever and never
// terminates.
```

Part XIII covers all of these in detail. The list here is so that the reader of this part knows the module was hard to get right.

#	Defect	How it was found
1	Infinite loop on zero-cost, zero-fuel cycles	Test suite hung; timeout after 5 minutes
2	Reference solver recorded the departure airport instead of the arrival airport	Cross-check against the primary solver
3	requiredAtDeparture under-fuelled the first departure	Cross-check against exhaustive search
4	The feasibility check demanded the whole route fit in the tanks	Cross-check against exhaustive search
5	A test expectation encoded the same bug as the code	The code was corrected and the test corrected with it

The honest lesson from defect 5

In one case the implementation and its test were wrong in exactly the same way, so the test passed and confirmed the bug. This is not a rare edge case — it is the normal failure mode of testing something against your own model of how it should behave. It is why AMS tests algorithms against *independent implementations* and *exhaustive search* rather than against expected values written by the same person who wrote the code.

Scale

```

it('plans a 400-airport network with 6,000 sectors in well under a second', () => {
  const N = 400;
  const flights = [];
  for (let i = 0; i < N; i += 1) {
    const degree = int(rand, 3, 18);
    for (let d = 0; d < degree; d += 1) {
      const to = int(rand, 0, N - 1);
      if (to === i) continue;
      flights.push({
        from: nodes[i], to: nodes[to],
        costCents: int(rand, 1_000, 900_000),
        // Fuel is uncorrelated with cost, so cost/fuel trade-offs are real.
        fuelKg: int(rand, 500, 12_000),
      });
    }
  }

  const t0 = performance.now();
  const result = solveRoute({
    flights, source: nodes[0], destination: nodes[N - 1],
    tankCapacityKg: 45_000, minReserveKg: 1_500,
  });
  const elapsed = performance.now() - t0;

  expect(result.feasible).toBe(true);
  expect(elapsed).toBeLessThan(1000);
});

```

The comment on fuel being uncorrelated with cost is deliberate. If fuel happened to correlate with cost in the generated data, the dominance rule would rarely need to keep more than one label per airport, and the interesting behaviour would never be exercised. Test data has to be built to be adversarial to the algorithm's shortcuts.

Part IX — The server side: money in a database

Everything so far has been calculation. This part covers the module that prevents an airline spending money it has not authorised — and the specific concurrency bug that makes it necessary.

```
/**
 * AMS — Cost Authority: the control core.
 *
 * THIS IS THE FUNCTION THAT PREVENTS AN AIRLINE OVERSPENDING.
 *
 * The problem, stated plainly:
 *
 * An airline's finance department approves purchase orders. Its
 * procurement department raises them. Its warehouse receives goods. Its
 * treasury releases money. Each department is necessary, and each
 * department is also, individually, capable of committing the airline
 * to expenditure the board never authorised.
 *
 * The classic failure is not a single large fraud. It is a hundred
 * individually-reasonable approvals, each taken against a budget line
 * that looked like it had room, which together exceed the board's
 * approved plan by a quarter. Each approver was honest. The aggregate
 * was not authorised.
 *
 * That happens because availability checks are TOCTOU: check the
 * balance, then write the commitment, and between those two operations
 * another request does the same thing. Two requests both see US$10,000
 * available. Both approve. The airline is US$10,000 overspent and nobody
 * was at fault in any individual transaction.
 *
 * THE FIX
 *
 * The read and the write happen inside one database transaction, and the
 * row is locked before it is read:
 *
 *     SELECT ... FOR UPDATE
 *
 * PostgreSQL blocks the second transaction until the first commits or
 * rolls back. The second transaction then re-reads the balance and sees
 * the FIRST transaction's commitment already deducted. The race is not
 * defended against; it is made unrepresentable.
 *
 * @module modules/finance/cost-authority
 */
```

backend/src/modules/finance/cost-authority.js

What a server is

The modules in Parts II to VIII are pure calculations: you give them data, they give you an answer, nothing is stored. A *server* is the part that listens for requests, calls those calculations, and records the results in a database.

```
// AMS currently has NO server. The pure domain modules exist and are
// fully tested; the HTTP layer, authentication and frontend are not yet
// built. What follows is the one backend module that is written, which is
// the control that matters most.
```

Asynchronous JavaScript and promises

Database operations take time. While one is waiting, the server must continue handling other requests. JavaScript handles this with *promises* — a value that will arrive later — and `async/await`, which makes waiting look sequential.

```
export async function lockBudgetLine(db, tenantId, lineId) {
  const line = await db('budget_lines')
    .where({ id: lineId, tenant_id: tenantId })
    .forUpdate()
    .first();

  if (!line) {
    throw new CostAuthorityError(
      `Budget line ${lineId} not found`, 'BUDGET_LINE_NOT_FOUND', { lineId },
    );
  }
  if (line.status !== 'ACTIVE') {
    throw new CostAuthorityError(
      `Budget line ${lineId} is ${line.status}, not ACTIVE`, 'BUDGET_LINE_INACTIVE', { lineId },
    );
  }
  if (line.is_frozen) {
    throw new CostAuthorityError(
      `Budget line ${lineId} is frozen by treasury`, 'BUDGET_LINE_FROZEN', { lineId },
    );
  }
  return /** @type {BudgetLine} */ (snakeToCamel(line));
}
```

backend/src/modules/finance/cost-authority.js

`await` means "wait for this to finish before continuing". The three checks after it are sequential and each one rejects a different failure: a line that does not exist, one that is closed, and one that treasury has frozen.

```
export class CostAuthorityError extends Error {
  constructor(message, code, details = {}) {
    super(message);
    this.name = 'CostAuthorityError';
    this.code = code;
    this.details = details;
    this.httpStatus = 422;
  }
}
```

A custom error class. Note it carries a machine-readable code, structured details, and an HTTP status. This is what lets a caller distinguish "insufficient funds" from "line frozen" without parsing the message text.

Why structured error codes matter to you

When something goes wrong in AMS, the response carries a code such as BUDGET_EXCEEDED and a shortfallCents figure. That means a supervisor reading a failed approval sees exactly how much money was missing, rather than having to infer it from a message. Auditable errors are a product feature, not just a developer convenience.

Database queries as chainable objects

```
await db('commitments')
  .insert({
    tenant_id: tenantId,
    budget_line_id: lineId,
    amount_cents: amountCents,
    movement_type: 'COMMITMENT',
    direction: 'DEBIT',
    available_before_cents: availableBefore,
    available_after_cents: availableBefore - amountCents,
    actor_id: actorId,
    created_at: db.fn.now(),
  })
  .returning(['id']);
```

Knex. The query is built by chaining method calls; nothing is sent to the database until the chain ends with await.

Call	Meaning
db('commitments')	start a query against this table
.where({ ... })	filter rows on these columns
.insert({ ... })	add a row with these values
.returning(['id'])	give me the new row's id back
.increment('committed_cents', amount)	add to a column by an amount — one atomic database operation
.decrement(...)	subtract from a column
.first()	return one row instead of an array
.forUpdate()	lock the row until the transaction ends
db.fn.now()	let the database supply the current timestamp

Why the database supplies the timestamp

db.fn.now() means the value comes from the database server, not from the application. If the application clock were wrong or the servers disagreed, two entries could be recorded out of order — which would corrupt the audit chain in Part X. Taking the timestamp from the system of record removes that possibility.

The race that overspends an airline

This is worth walking through slowly, because it is invisible in normal testing.

```
// THE BUG (no lock):

async function authoriseCostUnsafe(db, { tenantId, lineId, amountCents }) {
  const line = await db('budget_lines').where({ id: lineId }).first();
  const available = line.budgetedCents - line.committedCents;

  if (available < amountCents) throw new CostAuthorityError('Insufficient', 'BUDGET_EXCEEDED');

  // ---- another request arrives HERE, sees the same `available` ----
  await db('budget_lines').where({ id: lineId }).increment('committed_cents', amountCents);
  return { ok: true };
}
```

Two requests arrive at the same moment. Request A reads the line and sees US\$10,000 available. Before A writes anything, request B reads the *same* line and sees the *same* US\$10,000 available — because A has not written yet. Both approve US\$10,000. The airline is now committed to US\$20,000 against US\$10,000 of authority.

```
// The window between check and write is where the money disappears.
// A test that runs one request at a time will NEVER see this. It requires
// genuine concurrency to reproduce.
```

Why this class of bug survives code review

Sequential testing cannot find it. Code review cannot find it, because each individual statement is correct. Manual testing cannot find it, because it needs two requests in the same millisecond. It appears under production load, occasionally, at peak approval times. This is the strongest argument in this guide for the testing approach in Part XII: some failures are only reachable by construction, and the fix is to make them unrepresentable rather than to test for them.

Row locks and transactions

```
// ---- THE CRITICAL SECTION -----
// FOR UPDATE serialises every concurrent authorisation against this
// line. The second transaction waits here, then reads the balance the
// FIRST transaction left behind. Double-spend is unrepresentable.
// -----
const line = await lockBudgetLine(db, tenantId, lineId);

const availableBefore = availabilityCents(line);

if (availableBefore < amountCents) {
  throw new CostAuthorityError(
    `Insufficient cost authority: ${availableBefore} cents available, ${amountCents} cents requested`,
    'BUDGET_EXCEEDED',
    {
      lineId,
      availableCents: availableBefore,
      requestedCents: amountCents,
      shortfallCents: amountCents - availableBefore,
    },
  );
}
```

backend/src/modules/finance/cost-authority.js — the fix, in three lines.

The same race, with the fix. Request A locks the row and reads US\$10,000. Request B tries to lock the same row and **blocks** — it cannot proceed. A writes its commitment and commits. Only then does B acquire the lock and read — and it now sees US\$0 available, because A's commitment is already recorded. B correctly refuses.

```
/**
 * Authorise expenditure against a budget line, creating a commitment.
 *
 * MUST be called inside a transaction. The caller is responsible for
 * committing or rolling back; this function deliberately does not open
 * its own transaction, because the commitment write and the audit entry
 * must land atomically together.
 */
```

Why the function does not open its own transaction

A commitment and its audit entry must both happen or neither must. If this function opened and closed its own transaction, the audit entry would be written in a separate transaction that could fail independently — leaving a commitment with no audit trail, which is precisely the record an auditor would ask about. By refusing to open a transaction, the function makes the caller take responsibility for atomicity. A design that cannot be misused by accident.

Availability arithmetic and the decision to fail loudly

```

export function availabilityCents(line) {
  const frozen = line.frozenCents ?? 0;
  const available =
    line.budgetedCents + line.revisedCents + line.releasedCents
    - line.committedCents - line.pendingCents - line.expendedCents - frozen;

  if (available < 0) {
    throw new CostAuthorityError(
      `Budget line ${line.id} has negative availability (${available} cents). ` +
      'This is a control breach, not a data error.',
      'AVAILABILITY_NEGATIVE',
      { lineId: line.id, availableCents: available },
    );
  }
  return available;
}

```

Component	Meaning
budgetedCents	what the board approved
revisedCents	a later authorised revision (up or down)
releasedCents	funds formally released for spending
committedCents	approved but not yet paid
pendingCents	submitted, awaiting approval
expendedCents	paid
frozenCents	ring-fenced by treasury

The comment explains why frozen is separate from released: "a treasury ring-fence (a security deposit, a disputed supplier claim under litigation) removes spendable money WITHOUT creating a commitment. Conflating the two makes the reconciliation to the general ledger impossible."

Why a negative balance throws instead of clamping to zero

The comment says: "A negative result is a control breach, never a value to clamp. Clamping hides the breach; AMS throws and alerts." A negative availability means money was committed beyond what was authorised — either a bug, a manual database edit, or a deliberate override. Silently clamping it to zero would make the system report a healthy budget while the breach continues. This is the single most important cultural decision in the backend.

Idempotency: making retries safe

```
// Idempotency: a retried request must not commit twice.
if (params.idempotencyKey) {
  const prior = await db('commitments')
    .where({ tenant_id: tenantId, idempotency_key: params.idempotencyKey })
    .first();
  if (prior) {
    return {
      commitmentId: prior.id,
      availableBeforeCents: prior.available_before_cents,
      availableAfterCents: prior.available_after_cents,
      replayed: true,
    };
  }
}
```

Networks fail. A request may succeed on the server while the response is lost on the way back, and the client will retry. Without protection, a retry commits the money a second time.

```
return {
  commitmentId: String(created.id),
  availableBeforeCents: availableBefore,
  availableAfterCents: availableBefore - amountCents,
  replayed: false,
};
```

The idempotency key solves this. The client generates a unique value for the logical operation and sends it with every retry. If the server sees that key already committed, it returns the original result rather than performing the work again. The replayed flag tells the caller which happened.

Why this matters beyond technical correctness

The alternative — charging a customer twice because their connection dropped — is not a bug report, it is a complaint. Idempotency is what separates a system that can be relied on from one that generates occasional inexplicable failures that support has to explain.

All-or-nothing across a flight leg

```

export async function authoriseFlightLeg(db, { tenantId, lines, actorId }) {
  // Lock in a stable order to prevent deadlock between two rotations that
  // draw on overlapping budget lines in opposite order.
  const ordered = [...lines].sort((a, b) => a.lineId.localeCompare(b.lineId));

  const commitments = [];
  const failures = [];

  // Pass 1: validate every line under its lock before writing anything.
  for (const { lineId, amountCents } of ordered) {
    try {
      const line = await lockBudgetLine(db, tenantId, lineId);
      const available = availabilityCents(line);
      if (available < amountCents) {
        failures.push({ lineId, availableCents: available, requestedCents: amountCents });
      }
    } catch (err) {
      failures.push({ lineId, error: /** @type {CostAuthorityError} */ (err).code ?? 'UNKNOWN' });
    }
  }

  // Any failure aborts the whole leg. All-or-nothing, because a flight
  // cannot be half-authorised.
  if (failures.length > 0) {
    return { authorised: false, commitments: [], failures };
  }

  // Pass 2: all lines cleared – write the commitments.
  for (const { lineId, amountCents } of ordered) {
    commitments.push(await authoriseCost(db, { tenantId, lineId, amountCents, actorId }));
  }

  return {
    authorised: true, commitments, failures: [],
    totalAuthorisedCents: sum(...commitments.map((c) => c.availableAfterCents)),
  };
}

```

backend/src/modules/finance/cost-authority.js

Two design decisions here, both significant.

```

* A flight draws on several budget lines at once: fuel, crew, navigation,
* airport charges, catering. Each line must be checked, and the WHOLE leg
* must be rejected if any line fails – a partially authorised aircraft
* rotation is an aircraft that will be dispatched without fuel it was
* supposed to have.

```

- **Validate everything before writing anything.** Pass 1 checks every line without writing. Pass 2 writes only after all checks pass. Writing during validation would leave partial commitments when a later line fails.
- **Lock in a sorted order to prevent deadlock.** If two rotations draw on the same budget lines in opposite orders, they can wait on each other forever. Sorting by line id means every transaction acquires locks in the same sequence, which makes deadlock impossible.

Deadlock is not a hypothetical

Deadlock is one of the two classic database concurrency failures (the other being the race above). It happens when two transactions each hold a resource the other needs. The fix here is cheap — sort the lock order — and the comment says exactly why it is there. When reviewing database code, "in what order are locks acquired?" is a question worth asking every time.

The settlement rule that prevents double-spend

```
/**
 * Settle a commitment against a payment.
 *
 * NOTE: this reduces committed and increases expended by the same amount.
 * It does NOT restore availability. The money has left. Restoring
 * availability would let the same funds be committed a second time,
 * which is the exact failure this module exists to prevent.
 */
```

```
return { commitmentId, committedReleasedCents: amountCents, availabilityRestored: false };
```

A plausible-sounding mistake would be to return the money to the available balance on settlement. The comment explains why that would be catastrophic: the money has actually left the bank account. Releasing it would allow the same funds to be committed again.

```
/**
 * Release the difference when a supplier invoices less than ordered, or
 * when a purchase order is cancelled before delivery. Availability IS
 * restored, because no money left.
 */
```

The contrast between those two functions is the whole lesson. Money that left restores nothing; money that never left releases cleanly. Both rules are stated in comments immediately above the code.

```
export function freezeBudgetLine(db, { tenantId, lineId, amountCents, actorId, reason }) {
  if (!reason || reason.length < 10) {
    throw new CostAuthorityError(
      'A freeze requires a substantive reason of at least 10 characters; it is a '
      + 'governance action that appears in the audit pack',
      'FREEZE_REASON_REQUIRED', { lineId },
    );
  }
  // ...
}
```

A ten-character minimum on a reason field

This is a small piece of control design worth understanding. A freeze removes money from circulation, so it appears in the audit pack. Requiring a substantive written reason means the audit pack contains an explanation, not an unexplained number. It is a trivial piece of code expressing a governance principle: actions that affect money must be justifiable in words as well as in figures.

Part X — The audit trail

An airline audit trail is evidence presented to a statutory auditor, a lessor's technical representative, and a foreign civil aviation authority. If the airline can edit its own history, none of that evidence is worth anything.

```
/**
 * AMS — Tamper-evident audit chain.
 *
 * WHY THIS EXISTS
 *
 * An airline audit trail is evidence presented to a statutory auditor, to
 * a lessor's technical representative, and to a foreign civil aviation
 * authority inspecting the operator under the AOC. If the airline can
 * edit its own history, none of that evidence is worth anything.
 *
 * A conventional audit log — even append-only at the application level —
 * is editable by anyone holding database write access. Application-level
 * append-only is a policy; this is a mechanism.
 *
 * HOW IT WORKS
 *
 * Each entry stores the SHA-256 of its own content concatenated with the
 * previous entry's hash:
 *
 *     entry_hash = SHA256(prev_hash || canonical_entry)
 *
 * Any modification, deletion or reordering of an historic entry breaks
 * the chain at that point and every point after it. Recomputing the
 * chain detects it. Rewriting the whole chain requires the database
 * write key, which in production is not the same credential the
 * application runs with, and the chain head is additionally exported for
 * external anchoring.
 *
 * This is the same construction used for public ledgers, applied to
 * internal financial audit. It does not make the data trustworthy — it
 * makes tampering DETECTABLE, which is the property that matters.
 *
 * @module audit/chain
 */
```

Hashing: a fingerprint for data

```
import { createHash } from 'node:crypto';

/** Genesis sentinel. Chaining starts here. */
export const GENESIS_HASH = '0'.repeat(64);

export function computeHash(entry, prevHash) {
  const canonical = canonicalise({
    id: entry.id,
    tenantId: entry.tenantId,
    actorId: entry.actorId,
    entityType: entry.entityType,
    entityId: String(entry.entityId),
    action: entry.action,
    before: entry.before,
    after: entry.after,
    ip: entry.ip,
    occurredAt: entry.occurredAt,
  });
  return createHash('sha256').update(prevHash, 'utf8').update(canonical, 'utf8').digest('hex');
}
```

backend/src/audit/chain.js

A hash takes any input and produces a fixed-length fingerprint. Change one character anywhere in the input and the fingerprint changes completely. You cannot work backwards from the fingerprint to the original, and you cannot find a different input with the same fingerprint.

```
createHash('sha256').update('budget line 1: 1000000').digest('hex')
// 'a3f5...' (64 hexadecimal characters)

createHash('sha256').update('budget line 1: 1000001').digest('hex')
// '7b91...' – completely different, and there is no relationship between them
```

What this does and does not prove

The module comment is explicit: "It does not make the data trustworthy — it makes tampering DETECTABLE, which is the property that matters." Someone with write access could still rewrite the whole chain from scratch. What they cannot do is alter one entry and leave the rest consistent. The protection is against realistic partial tampering, not against an attacker who owns the database.

Chaining entries together

```
*   entry_hash = SHA256(prev_hash || canonical_entry)
*
* Any modification, deletion or reordering of an historic entry breaks
* the chain at that point and every point after it.
```

Each entry includes the *previous* entry's hash in its own calculation. This is the key idea, and it is what makes the chain a chain rather than a list of independent fingerprints.

```
Entry 1: hash(prev = 0000...0, content = "line 1 set to $10,000") → 8f2a...
Entry 2: hash(prev = 8f2a..., content = "line 1 set to $20,000") → 4c71...
Entry 3: hash(prev = 4c71..., content = "line 1 frozen") → b93d...
```

```
// Someone edits Entry 1 to say $50,000 and recomputes its own hash.
// Entry 2's stored prev-hash no longer matches. Verification fails at 2.
```

Why this stops the realistic attack

An ordinary database operator who wants to hide a single improper transaction edits one row. That row's hash no longer matches what entry 2 recorded as its predecessor, so the break is detected at entry 2. To hide it they must recompute every subsequent hash — and the chain head will differ from whatever value was externally anchored. This is the same construction used in public blockchain ledgers, applied to an internal financial audit.

Canonical form: making hashing reliable

```
/**
 * Deterministic JSON serialisation.
 *
 * Object key order must be stable or the hash is meaningless — the same
 * logical entry would hash differently depending on insertion order, and
 * verification would report corruption that never happened.
 */
```

```
export function canonicalise(value) {
  if (value === null || typeof value !== 'object') return JSON.stringify(value ?? null);
  if (Array.isArray(value)) return `[${value.map(canonicalise).join(',')}]`;
  const keys = Object.keys(** @type {Record<string, unknown>} */ (value)).sort();
  const parts = keys.map((k) => `${JSON.stringify(k)}:${canonicalise(** @type {any} */ (value)[k])}`);
  return `{${parts.join(',')}`;
}
```

backend/src/audit/chain.js

This function solves a problem that looks trivial and is not. Two objects with the same content can have their fields in different orders, and JavaScript preserves insertion order when you convert to text.

```
canonicalise({ b: 2, a: 1 }) // '{"a":1,"b":2}'
canonicalise({ a: 1, b: 2 }) // '{"a":1,"b":2}' ← identical, by design
```

A bug that would have been catastrophic and invisible

Without sorting, the first object hashes differently from the second. Verification would report corruption on a chain that is perfectly intact. The failure mode is the worst kind: the system reports a problem that does not exist, and engineers spend weeks investigating a phantom. This is a small amount of code preventing a large amount of wasted time.

Verifying a chain

```
export function verifyChain(entries) {
  let prev = GENESIS_HASH;
  for (const entry of entries) {
    const expected = computeHash(entry, prev);
    if (expected !== entry.entryHash) {
      return {
        valid: false,
        entriesChecked: entries.indexOf(entry),
        brokenAt: entry.id,
        reason: 'Hash mismatch: entry content or ordering has been altered',
      };
    }
    prev = entry.entryHash;
  }
  return { valid: true, entriesChecked: entries.length, brokenAt: null, reason: null };
}

export function verifyTenantChain(allEntries, tenantId) {
  const scoped = allEntries
    .filter((e) => e.tenantId === tenantId)
    .sort((a, b) => a.id - b.id);
  return { tenantId, ...verifyChain(scoped) };
}

export function chainHead(entries) {
  if (entries.length === 0) return GENESIS_HASH;
  const last = [...entries].sort((a, b) => a.id - b.id).at(-1);
  return last.entryHash;
}
```

Verification walks the chain recomputing each hash. The first entry whose stored hash does not match its recomputed value is reported by identifier, so an investigator knows exactly where the history diverges.

```
// verifyTenantChain exists so one tenant's corruption cannot mask or
// contaminate another's. AMS is multi-tenant: each airline is a separate
// customer with separate data. An audit must be provable per customer.
```

Anomalies beyond hash mismatch

```

export function auditAnomalies(entries) {
  const findings = [];
  const ordered = [...entries].sort((a, b) => a.id - b.id);

  ordered.forEach((e, i) => {
    if (i > 0 && e.id <= ordered[i - 1].id) {
      findings.push({ severity: 'SEV1', code: 'OUT_OF_ORDER', entryId: e.id, detail: 'Non-monotonic entry id' });
    }
    if (!e.actorId || e.actorId === 'system') {
      findings.push({ severity: 'INFO', code: 'SYSTEM_ACTOR', entryId: e.id, detail: 'Entry written by system actor' });
    }
    if (e.action === 'DELETE' && e.before === null) {
      findings.push({ severity: 'SEV2', code: 'DELETE_WITHOUT_SNAPSHOT', entryId: e.id, detail: 'Delete recorded without snapshot' });
    }
    if (e.action === 'APPROVE' && e.entityType === 'cost_authority'
      && String(e.actorId) === String(/** @type {any} */ (e.before)?.approved_by ?? '')) {
      findings.push({ severity: 'SEV1', code: 'SELF_APPROVAL', entryId: e.id, detail: 'Approver matched the author' });
    }
  });

  return {
    findings,
    highestSeverity: findings.reduce(
      (worst, f) => (worst === null || rank(f.severity) < rank(worst) ? f.severity : worst),
      /** @type {string|null} */ (null),
    ),
  };
}

```

Hash verification answers "was this changed?". Anomaly detection answers "was this a legitimate thing to do?" Both matter, and they catch different problems.

Finding	Severity	What it suggests
OUT_OF_ORDER	SEV1	Entries were reordered or rewritten
SELF_APPROVAL	SEV1	The same person authorised what they requested — a segregation-of-duties breach
DELETE_WITHOUT_SNAPSHOT	SEV2	A deletion was recorded with no record of what was deleted
SYSTEM_ACTOR	INFO	The entry came from an automated process, not a person

```

export const SOD_RULES = [
  'SELF_APPROVAL', // approver === requester
  'EVALUATOR_IS_AUTHOR', // evaluator also authored the sourcing event
  'PAYMENT_RELEASER_IS_ORIGINATOR',
  'SUPPLIER_BIDS_OWN_TENDER',
  'TECHNICAL_APPROVES_OWN_CHECK',
  'PERIOD_CERTIFIER_WITHOUT_AUTHORITY',
];

```

shared/src/domain.js — the separation-of-duties rules the anomaly detector looks for.

Part XI — Types without TypeScript

AMS is written in plain JavaScript with no compilation step. This part explains what safety that actually buys, and where the gaps are.

The decision to write plain JavaScript

```
## The JavaScript decision (ADR-001)
```

```
No TypeScript build. Correctness is carried by Zod validation at every request boundary, JSDoc annotations, and a `checkJs` **no-emit** gate in CI. The gate found **47 genuine defects** on first run against this codebase – including two source bugs (`cyclesPerHour` undefined; an unreachable branch in `routeVerdict`) that the 118 unit tests missed.
```

```
That is the argument for keeping it. Revisit if the team exceeds four engineers or the codebase passes 250,000 lines.
```

Two corrections to the ADR as written

First, Zod is **not installed and not used anywhere** in this codebase. That sentence overstates what exists. Second, the test count is now 190 and there is **no CI pipeline configured** for this project — the gate can be run locally with `npm run typecheck` but nothing enforces it automatically. Both claims should be corrected before this document is used to justify the decision to a reviewer.

The underlying argument is still sound, and the trigger conditions are sensible. Plain JavaScript with an automated type check is a legitimate choice for a team that wants no build step. What matters is that the safety mechanisms claimed actually exist and run.

What JSDoc does and does not do

```
/**
 * @typedef {Object} BudgetLine
 * @property {string} id
 * @property {string} tenantId
 * @property {string} accountId
 * @property {string} costCenterId
 * @property {string} fundId
 * @property {string} vehicleRegistration For fuel and maintenance budgets
 * @property {number} fiscalYear
 * @property {string} categoryId
 * @property {number} budgetedCents
 * @property {number} revisedCents
 * @property {number} releasedCents
 * @property {number} committedCents
 * @property {number} pendingCents Submitted but not yet approved requisitions
 * @property {number} expendedCents
 * @property {number} [frozenCents] Ring-fenced by treasury (security deposits, disputed claims)
 * @property {boolean} isFrozen
 * @property {string} status ACTIVE | REVISED | CLOSED
 */
```

Read this as a data dictionary rather than as code. It documents what a budget line record contains, what type each field is, which fields are optional (written in square brackets), and — in the trailing comments — what each one means in business terms.

Annotation	Meaning
@param {number} amountCents	a parameter named amountCents, which must be a number
@returns {Cents}	the return value is integer cents
@returns {Cents[]}	an array of integer cents
@property {number} [frozenCents]	an optional field; may be absent
@returns {number null}	a number, or explicitly nothing
{readonly [number, number]}	a fixed-length array
{Record<string, number>}	an object with string keys and number values

```
/**
 * @typedef {Object} FuelBurn
 * @property {number} upliftKg
 * @property {number} takeoffFuelKg   Fuel on board at brake release
 * @property {number} landingFuelKg   Fuel remaining at touchdown
 */
```

Those trailing comments are the most valuable part

takeoffFuelKg and landingFuelKg are self-explanatory. "Fuel on board at brake release" and "fuel remaining at touchdown" remove all ambiguity about *when* the measurement is taken — and getting that wrong produces a plausible-looking wrong number. This is documentation doing real work.

The type-check gate

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true,
    "noEmit": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": false,
    "skipLibCheck": true
  },
  "include": ["shared/src/**/*.js", "backend/src/**/*.js", "tests/**/*.js"],
  "exclude": ["node_modules"]
}
```

Setting	What it does	Why
allowJs	permit JavaScript files	the whole point

Setting	What it does	Why
checkJs	type-check JavaScript without renaming it	safety without a build
noEmit	never write output files	nothing is compiled, so nothing can be emitted
target	assume modern JavaScript	??, optional chaining, at () are available
skipLibCheck	do not check library type definitions	faster, and third-party types are not our concern
strict: false	relaxed null checking	the known gap — see below

```
npm run typecheck    # runs tsc --noEmit; fails on any type error
npx tsc --noEmit    # the same thing directly
```

The strict:false gap, stated plainly

With `strict: false`, TypeScript will not complain if a function that expects a value is handed `null` or `undefined`. In a codebase that handles money and airworthiness, that is the wrong default. `strict: true` would likely surface real defects on first run. This is a known, recorded weakness rather than an oversight.

Where the guarantee really comes from

```
* Union types
*
* These are JSDoc-only. In a plain-JavaScript codebase there is no
* `export type` syntax, so union types are documented for the editor
* and for `checkJs`, and enforced at RUNTIME by the CHECK constraints
* generated on each state column from the const arrays above.
*
* A value that is not in the corresponding array cannot reach the
* database. The type annotation is a convenience; the constraint is
* the guarantee.
*/
```

shared/src/domain.js — this paragraph is the clearest statement of the project's safety philosophy.

Read that twice. It says: *annotations are a convenience, constraints are the guarantee*. A JSDoc comment can be wrong and nothing breaks. A database CHECK constraint cannot be violated, whatever any application code does.

```
/**
 * @typedef {'A'|'B'|'C'|'D'} MelCategory
 */
export const MEL_CATEGORIES = {
  A: { days: null, note: 'No standard interval specified; starts at deferral' },
  B: { days: 3, note: '3 calendar days excluding the day of discovery' },
  C: { days: 10, note: '10 calendar days excluding the day of discovery' },
  D: { days: 120, note: '120 calendar days excluding the day of discovery' },
};
```

```
if (!spec) throw new TypeError(`Unknown MEL category "${category}"`);  
if (spec.days === null) return null; // Category A: no standard interval
```

The lookup `MEL_CATEGORIES[category]` returns undefined for an invalid category, and the very next line throws. This is the pattern throughout AMS: a JavaScript-level guard, backed by a database-level constraint. Two independent mechanisms, one of which cannot be bypassed.

The three-layer safety model, in one paragraph

Layer one: JSDoc annotations and the type-check gate catch mistakes before the code runs. Layer two: explicit runtime checks (`if (!spec) throw`) catch them at execution. Layer three: database constraints catch them regardless of what the application does. No single layer is trusted alone. This is the actual answer to "is plain JavaScript safe for this?" — not that the language is safe, but that the design does not depend on the language being safe.

Part XII — Testing as the real safety net

Given that Part XI showed the type annotations are a convenience rather than a guarantee, the tests are where safety actually lives. This part explains what the AMS test suite does and, more importantly, what it deliberately does not do.

What a test suite is for

```
/**
 * Verification suite for two optimisation algorithms.
 *
 * These tests do not assert what the code intends to do. They assert
 * agreement between three INDEPENDENT implementations, plus structural
 * properties that must hold for any correct answer.
 *
 * 1. solveRoute      — Pareto label-setting (production)
 * 2. solveRouteDP    — fuel-indexed DP (reference)
 * 3. bruteForceRoute — exhaustive enumeration (oracle)
 *
 * If all three agree on thousands of random instances, and the oracle is
 * correct by construction, the production algorithm is correct on those
 * instances. That is meaningfully stronger than a test suite written
 * from the same mental model as the implementation — which is how the
 * `_roll` defect shipped through 118 green tests.
 */
```

tests/routing.test.js — the file header. Note the last sentence: it explicitly names the failure mode the suite is designed to avoid.

Structural tests versus expected values

```
describe('defect 1 — no fabricated metrics reach the API', () => {
  it('the aggregate carries no internal or underscore-prefixed fields', () => {
    const agg = aggregatePnl([LEG(150), LEG(160)]);
    const offenders = Object.keys(agg).filter((k) => k.startsWith('_'));
    expect(offenders).toEqual([]);
  });

  it('every aggregate metric is a real sum of real flight facts', () => {
    const flights = [LEG(150), LEG(160), LEG(140)];
    const agg = aggregatePnl(flights);
    // ...
  });
});
```

tests/costing-defects.test.js

There are two ways to test. The common one is to write down what you expect the answer to be and check it. That only proves the code does what you believed it should do.

The stronger one asserts a *property* that must hold for any correct implementation. "The aggregate has no underscore-prefixed fields" is that kind of test. It would have failed the `_roll` defect, and it would have failed it for the right reason — the defect was that a fabricated field was being returned at all.

Why property tests catch more

A property test survives changes you have not thought about. If someone later adds a genuinely useful `_internal` field, an expected-value test still passes but the property test flags it for review. Conversely, an expected-value test has to be rewritten every time a correct change alters a number, and the temptation under time pressure is to update the expectation rather than investigate.

Testing against three implementations

```
it('agrees with the exhaustive oracle on 3000 random instances', () => {
  let compared = 0;
  let feasible = 0;
  for (let seed = 1; seed <= 3000; seed += 1) {
    const rand = mulberry32(seed * 2654435761);
    const n = int(rand, 2, 6);
    const inst = randomInstance(rand, n);
    if (inst.flights.length === 0) continue;

    const pareto = solveRoute(inst);
    const oracle = bruteForceRoute(inst);

    compared += 1;
    if (oracle.feasible) feasible += 1;

    expect(pareto.feasible, `seed ${seed}: feasibility must match oracle`).toBe(oracle.feasible);
    if (oracle.feasible) {
      expect(pareto.costCents, `seed ${seed}: optimal cost must match oracle`).toBe(oracle.costCents);
      const { cost } = assertRouteLegal(inst, pareto.stops);
      expect(cost, `seed ${seed}: reported cost must equal the sum of its own legs`).toBe(pareto.costCents);
    }
  }
  expect(compared).toBeGreaterThan(2500);
  expect(feasible / compared).toBeGreaterThan(0.3);
});
```

Three assertions do different work in this test. The first says the two solvers must agree on feasibility. The second says they must agree on the optimal cost. The third — and this is the one most teams skip — verifies the returned route is internally consistent, by walking its own legs and re-adding the cost.

```
/** Recompute cumulative fuel for a route and assert it is legal. */
function assertRouteLegal(instance, stops) {
  const index = new Map();
  for (const f of instance.flights) index.set(`${f.from}>${f.to}`, f);
  const usable = instance.tankCapacityKg - instance.minReserveKg;
  let cumulative = 0;
  let cost = 0;
  for (let i = 0; i < stops.length - 1; i += 1) {
    const leg = index.get(`${stops[i]}>${stops[i + 1]}`);
    expect(leg, `leg ${stops[i]}->${stops[i + 1]} must exist`).toBeDefined();
    cumulative += leg.fuelKg;
    cost += leg.costCents;
  }
  expect(cumulative).toBeLessThanOrEqual(usable);
  return { cumulative, cost };
}
```

Why the self-consistency check is not redundant

A solver could in principle return the correct cost alongside a route that does not achieve it — for instance if a reconstruction bug assembled the stops from the wrong predecessor pointers. Comparing two costs would pass. Walking the returned route and re-adding its own costs catches that class of bug, and independently confirms the fuel constraint holds on the answer that will actually be used.

Property tests: invariants that must hold

```
describe('monotonicity – properties any correct solver must satisfy', () => {
  it('more tank capacity never increases the optimal cost', () => {
    for (let seed = 1; seed <= 800; seed += 1) {
      const inst = randomInstance(mulberry32(seed * 1103515245 + 12345), int(rand, 3, 6));
      const base = solveRoute({ ...inst, tankCapacityKg: 20, minReserveKg: 3 });
      const bigger = solveRoute({ ...inst, tankCapacityKg: 45, minReserveKg: 3 });
      if (base.feasible && bigger.feasible) {
        expect(bigger.costCents, `seed ${seed}: capacity grew, cost must not rise`)
          .toBeLessThanOrEqual(base.costCents);
      }
    }
  });

  it('a lower mandatory reserve never increases the optimal cost', () => { /* ... */ });

  it('raising any sector cost never decreases the optimal cost', () => { /* ... */ });

  it('when the unconstrained optimum is feasible, it is also the constrained optimum', () => {
    let checked = 0;
    for (let seed = 1; seed <= 1200; seed += 1) {
      // ...
      if (result.costCents === unconstrained) checked += 1;
    }
    expect(checked).toBeGreaterThan(20);
  });
});
```

tests/routing.test.js

These tests know nothing about the algorithm. They express truths about the *problem* that any correct solution must respect, and they check thousands of instances. A completely different correct implementation would pass them.

Property	Intuition
More capacity cannot cost more	A bigger tank removes constraints; it cannot add them
A lower reserve cannot cost more	Same reasoning — a weaker requirement cannot hurt
Higher prices cannot cost less	Expensive flights are never an opportunity
Feasible unconstrained optimum = constrained optimum	If the cheap answer is allowed, it should win

Making random tests reproducible

```
expect(pareto.feasible, `seed ${seed}: feasibility must match oracle`).toBe(oracle.feasible);
expect(pareto.costCents, `seed ${seed}: optimal cost must match oracle`).toBe(oracle.costCents);
```

Every random assertion includes the seed in its failure message. When a test fails in CI, the log names the exact case, and the loop can be re-run with that seed to reproduce it. This is a small detail that decides whether a random failure is a five-minute fix or a two-day investigation.

Regression tests for real defects

```
/**
 * Regression tests for two defects found by REVIEW rather than by any
 * failing test. Both shipped green through 118 tests.
 *
 * DEFECT 1 – aggregatePnl computed a `roll` object from fabricated input
 * (distanceKm: 0, passengers: 0, a `seatsFrom` helper that multiplied by
 * an undefined field and therefore always returned 0). It produced an
 * object of near-zero metrics and attached it to the return value as
 * `_roll`, so it serialised into every route P&L API response.
 *
 * DEFECT 2 – breakEvenPassengers computed
 *   (revenue / X) * 0          ← multiplied by ZERO
 *   and fell back to a hardcoded US$120 fare x 0.62.
 * The number it returned was derived from nothing but a magic constant.
 *
 * The lesson these encode: a test suite that only asserts what the code
 * intends to do will never catch a function whose intent was never
 * implemented. These tests assert STRUCTURAL properties – no fabricated
 * fields reach the API, and no magic constant influences an answer.
 */
```

tests/costing-defects.test.js — the header is the most valuable part of this guide's case for how AMS tests.

What the tests do not prove

The most important limitation in this document

These tests prove the code does what the authors intended. They do **not** prove the intent was correct. A complete green suite on the routing module proves the Pareto solver is optimal *given that the reserve is capacity – reserve and given that a sector is flyable if cumulative fuel fits*. If the regulatory reserve requirement is wrong, every test still passes.

This is the boundary between what software can verify and what requires a qualified human. Part XIV returns to it, because for a system that handles financial and airworthiness decisions, that boundary is where the real risk lives.

Part XIII — Defects found, and what they teach

This part documents six real defects found in the AMS codebase, four of them by automated verification rather than by reading the code. They are included because a system that has never been wrong has not been tested — and because each one is a lesson about how to work.

Why a training document admits its own bugs

Because the alternative is worse. A guide that presented only working code would teach patterns without teaching how to recognise when they are broken. Each defect below was found by a specific mechanism, and that mechanism is the transferable lesson.

Defect 1: an object built from nothing

```
// WHAT THE CODE DID
aggregatePnl([...flights])._roll
// → { distanceKm: 0, passengers: 0, seats: 0, loadFactorPpm: 0, ... }

// A helper named seatsFrom multiplied by an undefined field,
// so it always returned 0. Every figure in _roll was therefore zero,
// and _roll was attached to the return value of aggregatePnl –
// which means it serialised into every route P&L API response.
```

Every number in that object was fabricated. It was not a mis-calculation; it was an object whose inputs were all zero because the code reading them was itself broken.

```
// THE FIX, AS A TEST
describe('defect 1 – no fabricated metrics reach the API', () => {
  it('the aggregate carries no internal or underscore-prefixed fields', () => {
    const agg = aggregatePnl([LEG(150), LEG(160)]);
    const offenders = Object.keys(agg).filter((k) => k.startsWith('_'));
    expect(offenders).toEqual([]);
  });

  it('every aggregate metric is a real sum of real flight facts', () => {
    const flights = [LEG(150), LEG(160), LEG(140)];
    const agg = aggregatePnl(flights);

    expect(agg.passengers).toBe(450);
    expect(agg.seatsOffered).toBe(567);
    expect(agg.distanceKm).toBe(33_000);
    expect(agg.blockHours).toBe(33);
  });
});
```

How this shipped through 118 green tests

No test asserted anything about `_roll`, because nobody had asked for it. The tests verified the fields they knew about, and the fabricated object sat in the response unnoticed until someone read the actual output.

Found by reading, not by testing.

The general lesson: if a value is being returned to a user or an API consumer, someone should ask where every field in it came from. A field prefixed with an underscore conventionally means "internal" — and this one was both internal and wrong.

Defect 2: an answer multiplied by zero

```
// WHAT THE CODE DID
function breakEvenPassengers(agg) {
  return (agg.revenueCents / agg.loadFactorPpm) * 0;    // always 0
}

// and when that produced something useless, it fell back to:
const HARDCODED_FARE = 12_000;
return HARDCODED_FARE * 0.62;                        // always 7440
```

This function answered a commercially critical question — how many extra passengers would this route need to break even — with a number derived from nothing. Multiplying by zero always gives zero; the hardcoded fare of US\$120 was the only thing that ever appeared in the output.

```
// THE FIX
function breakEvenPassengers(agg) {
  const flights = agg.flights || 1;
  const passengers = agg.passengers || 0;

  const revenuePerPassengerCents = passengers > 0
    ? Math.round(agg.revenueCents / passengers)
    : 0;
  const marginalCostCents = agg.marginalSeatCostCents ?? 0;
  const contributionPerPassengerCents = revenuePerPassengerCents - marginalCostCents;

  const perFlightLoss = Math.max(0, -agg.contributionCents) / flights;
  if (perFlightLoss === 0) {
    return { additionalPassengersPerFlight: 0, closeableByLoadFactor: true,
      revenuePerPassengerCents, contributionPerPassengerCents };
  }

  // Carrying a passenger costs more than they pay. More load does not
  // fix this; the route needs re-pricing, a fleet change or withdrawal.
  if (contributionPerPassengerCents <= 0) {
    return { additionalPassengersPerFlight: Number.MAX_SAFE_INTEGER,
      closeableByLoadFactor: false,
      revenuePerPassengerCents, contributionPerPassengerCents };
  }

  return {
    additionalPassengersPerFlight: Math.ceil(perFlightLoss / contributionPerPassengerCents),
    closeableByLoadFactor: true,
    revenuePerPassengerCents, contributionPerPassengerCents,
  };
}
```

```
/**
 * If that is positive, additional passengers CLOSE the gap and the answer
 * is the per-flight loss divided by it. If it is negative – the airline is
 * paying more to carry a passenger than that passenger earns – adding
 * passengers makes the loss WORSE, and the honest answer is that the gap
 * cannot be closed by load factor at all.
 *
 * An earlier version of this function multiplied by zero and fell back to
 * a hardcoded US$120 fare, so it returned a number derived from nothing.
 * A magic constant here produces a confidently wrong number to a manager
 * deciding a route's future.
 */
```

The fix is not just arithmetic — it adds a business case

The corrected version handles a case the original never could: what if a passenger costs more to carry than they pay? In that situation more load makes the loss *worse*, and the honest answer is that load factor cannot fix the problem. The function now returns `closeableByLoadFactor: false` and says so. **A function that can only ever produce a positive answer is usually hiding a case.**

Situation	Correct answer	What the old code said
Loss, passengers profitable	A real number of extra passengers	7440, always
Loss, each passenger costs more than they pay	Cannot be closed by load factor	7440
Already profitable	0	7440

The test that encodes the fix

The regression test asserts **structurally**: that the returned value is derived from the aggregate's real revenue-per-passenger and real marginal cost, and that changing those inputs changes the answer. A test asserting "the answer is 7440" would have passed the broken code and failed the fix.

Defect 3: the test that shared the bug

```
// THE BUG
export function requiredAtDeparture(legFuelKg, reserveKg) {
  for (let i = n - 1; i >= 0; i -= 1) {
    const forLeg = legFuelKg[i] + reserveKg;
    required[i] = i === n - 1 ? forLeg : Math.max(forLeg, required[i + 1]);
  }
}

// legs [1000, 2000, 500], reserve 300, route A-B-C-D
// This produced: required[2] = 800, required[1] = 2300, required[0] = 2300
// The correct answer:                2800                3800
```

The old formula used `max(leg + reserve, required[i+1])`. It should be `leg + max(reserve, required[i+1])`. These differ: you need enough to fly the leg *plus* whatever the rest of the route requires, whereas the old version took the larger of the two, which under-fuels the earlier departures.

```
// THE FIX
export function requiredAtDeparture(legFuelKg, reserveKg) {
  const n = legFuelKg.length;
  const required = new Array(n);
  for (let i = n - 1; i >= 0; i -= 1) {
    // Departing stop i you must burn leg i to reach stop i+1, and then
    // hold AT LEAST the reserve – but possibly more than the reserve,
    // because everything the rest of the route needs must still be on
    // board when you leave i. Hence max(), and hence the composition is
    // legFuel[i] + max(reserve, required[i+1]), not max(legFuel[i]+reserve,
    // required[i+1]). The latter silently under-fuels the first departure
    // of every multi-leg route.
    const neededBeyond = i === n - 1 ? reserveKg : Math.max(reserveKg, required[i + 1]);
    required[i] = legFuelKg[i] + neededBeyond;
  }
  return required;
}
```

The part that matters: the test was wrong too

The original test asserted `required[1]` is 2300 and `required[0]` is 2300 — exactly the buggy values. The implementation and its test were wrong in the same direction, so the test passed and confirmed the bug. It was caught only by the exhaustive oracle comparison.

```
// THE CORRECTED TEST, WITH THE REASONING WRITTEN DOWN
it('composes the whole remainder of the route, not just the next leg', () => {
  // legs [1000, 2000, 500], reserve 300, route A-B-C-D.
  // Departing C: 500 leg + 300 reserve           = 800
  // Departing B: 2000 leg + max(300, 800)         = 2800
  // Departing A: 1000 leg + max(300, 2800)         = 3800
  //
  // This test originally asserted 2300/2300, which was wrong in exactly
  // the same way the implementation was wrong: it treated the reserve
  // as if it were the only requirement beyond the next leg, so it
  // under-fuelled the first departure. Both the code and the test were
  // wrong in the same direction, which is what a test suite written from
  // the author's own mental model looks like. Caught by the exhaustive
  // oracle, not by inspection of either.
  const required = requiredAtDeparture([1000, 2000, 500], 300);
  expect(required[2]).toBe(800);
  expect(required[1]).toBe(2800);
  expect(required[0]).toBe(3800);
});
```

This is the most instructive defect in the codebase

A test written by the same person, from the same mental model, asserting the same misunderstanding. Testing something against your own model of how it should behave does not validate the model. This is why the AMS routing tests compare against an **exhaustive search** rather than against hand-written expectations — an oracle cannot share your misunderstanding, because it has no model at all.

Defect 4: the wrong node in the solver

```
// THE BUG
buckets[nf].push({ fuelUsed: nf, costCents: nc, from: state.from, prevFuel: f });
//
// Carried the node the flight DEPARTED from, not the one it ARRIVED at.

// Later, when that state was processed:
for (const edge of adjacency.get(state.from) ?? []) {
//
// re-expanded the WRONG airport

// The consequence: the DP reported routes as INFEASIBLE that the primary
// solver had found. Nobody noticed, because nothing compared the two.
```

```
// THE FIX
    buckets[nf].push({ node: edge.to, costCents: nc,
                      prevNode: state.node, prevFuel: f });
```

Why this is the strongest argument in the guide

A second implementation is only useful if something forces the two to agree. Two solvers that disagree and are never compared provide no more assurance than one. This defect sat in the code, passing every existing test, until the cross-check was added.

Defect 5: refusing the plans it existed for

```
// THE BUG
const required = requiredAtDeparture(legFuelKg, reserveKg);
if (required[0] > tankCapacityKg) {
    throw new RefuelError(`Route needs ${required[0]}kg at the first departure
        but tank capacity is ${tankCapacityKg}kg. The route is not flyable.`);
}
```

`requiredAtDeparture` returns the fuel needed to complete the *whole remaining route* without refuelling. For any multi-stop route that exceeds tank capacity — which is normal, because you refuel at the intermediate stops. The check therefore rejected every multi-stop route, which is precisely what the function exists to plan.

```
// THE FIX – check each LEG, not the whole route
for (let i = 0; i < legFuelKg.length; i += 1) {
    if (legFuelKg[i] + reserveKg > tankCapacityKg) {
        throw new RefuelError(
            `Sector ${stops[i]} -> ${stops[i + 1]} needs ${legFuelKg[i] + reserveKg}kg ` +
            `(leg + ${reserveKg}kg reserve) but tank capacity is only ${tankCapacityKg}kg. ` +
            'No refuelling schedule can make this route legal.',
            'REFUEL_ROUTE_INFEASIBLE',
        );
    }
}
}
```

The error message now names the offending sector

The fix does not just change the arithmetic — it improves the diagnosis. "Route needs 16500kg but tank capacity is 10000kg" told the user nothing actionable. "Sector B → C needs 30500kg" tells them exactly which leg cannot be flown and why.

Defect 6: an infinite loop in a legal case

```
// THE BUG
function dominates(ca, fa, cb, fb) {
  return ca <= cb && fa <= fb && (ca < cb || fa < fb);
}

// A zero-cost, zero-fuel cycle produces a label IDENTICAL to one already
// present. Identical labels do not strictly dominate one another — the
// `&& (ca < cb || fa < fb)` clause is false when both are equal.
// So the identical label was accepted, expanded, and produced another
// identical label, forever.
```

```
// THE FIX
for (const other of target) {
  if (dominates(other.costCents, other.fuelUsedKg, costCents, fuelUsedKg)) {
    dominated = true; break;
  }
  if (other.costCents === costCents && other.fuelUsedKg === fuelUsedKg) {
    dominated = true; break;
  }
}
```

```
// THE FIX, EXPLAINED IN THE CODE
// The duplicate check is not cosmetic. A zero-cost, zero-fuel cycle
// produces a label identical to one already present. Identical
// labels do not strictly dominate one another, so without this the
// planner re-queues the same (cost, fuel) pair forever and never
// terminates. Rejecting the duplicate is safe: the incumbent label
// has an equally good prefix and is already in the heap.
```

How it was found, and what that says about testing

The test suite **hung for five minutes and was killed by a timeout**. A correctness assertion would not have found this — the code was not producing wrong answers, it was producing no answers at all. Only running it with a time limit revealed the problem.

This is why the AMS test suite includes an explicit performance assertion and why edge cases like zero-cost cycles are tested at all. **Non-termination is a failure mode that correctness assertions cannot detect.**

What the pattern tells you

Defect	Type	Found by
1. Fabricated _roll object	Wrong data returned	Reading the code
2. Answer multiplied by zero	Wrong arithmetic	Reading the code
3. Under-fuelled first departure	Wrong formula	Exhaustive oracle comparison
4. Wrong node in the DP solver	Wrong state carried	Cross-check between two solvers
5. Feasibility check too strict	Wrong condition	Exhaustive oracle comparison
6. Infinite loop on identical labels	Non-termination	Test suite timeout

Four patterns run through all six.

1. **Two of six were found by a human reading code, not by any test.** The two costing defects had a fully green 118-test suite. No automated check was looking for the right things.
2. **Three were found by differential testing** — comparing an algorithm against an independent implementation or exhaustive search. This is the highest-yield technique in the codebase.
3. **One was found by the test simply running.** Non-termination is invisible to assertions.
4. **In two cases the test encoded the bug.** This is the most common failure mode of hand-written expectations and the reason differential testing exists.

The honest summary

The codebase went from 118 tests to 190, and the six defects above were found along the way. That is roughly one defect per 12 tests added. It is also why the routing module — the newest and most thoroughly tested part of the system — is the part the engineers are most confident in. Confidence tracks verification effort, and nothing else.

Part XIV — What comes next

This part covers planned work, the boundary of what software can verify, and a glossary of the terms used throughout.

The server layer, when it is built

AMS currently has no HTTP layer. The domain modules in `shared/src/` are complete and tested; what is missing is the code that receives requests, calls those modules, and writes results to the database.

```
// WHAT WOULD BE BUILT (does not exist yet)
import express from 'express';
import knex from 'knex';

const app = express();
const db = knex(knexConfig);

app.post('/api/tenants/:tenantId/flight-costs', async (req, res) => {
  // 1. authenticate — who is calling?
  // 2. validate — is the request well-formed?
  // 3. authorise — may this caller do this to this tenant's data?
  // 4. compute — call flightPnl() from shared/src/costing.js
  // 5. persist — write the attributed cost, in a transaction
  // 6. audit — record who did what, in the same transaction
});
```

The dependency order is not negotiable

Steps 1 to 3 come before any data is read or written. Building the reporting and export features on top of an API with no authentication would put unauthenticated access to financial data behind a public URL. The domain logic is ready; the door is not.

Identity, access and multi-tenancy

AMS is a multi-tenant platform: multiple airlines, each with separate data. This is not a feature bolted on at the end — it changes the database schema, every query, and the audit model.

```
export const ROLES = [
  'platform_admin', 'tenant_admin', 'ceo', 'finance_director',
  'controller', 'accountant', 'procurement_manager',
  'technical_manager', 'flight_ops_manager', 'safety_officer', 'auditor',
];

export const DEPARTMENTS = [
  'executive', 'flight_operations', 'technical_engineering', 'crew_scheduling',
  'training_standards', 'safety_risk', 'quality_assurance',
  'commercial_revenue', 'ticketing_distribution', 'customer_service',
  'cargo', 'ground_operations', 'finance_treasury', 'procurement_supply',
  'human_resources', 'it_mis',
];
```

```
// Tenant isolation appears in every query in cost-authority.js:
const line = await db('budget_lines')
  .where({ id: lineId, tenant_id: tenantId }) // ← always both columns
  .forUpdate()
  .first();

// And in audit verification, so one tenant's data cannot contaminate another's:
export function verifyTenantChain(allEntries, tenantId) {
  const scoped = allEntries.filter((e) => e.tenantId === tenantId);
  return { tenantId, ...verifyChain(scoped) };
}
```

```
* Aviation regulations mandate named accountable managers, so the
* department structure is a regulatory artefact, not a UI preference.
*
* 16 registered. 5 functional in wave 1: Finance & Treasury,
* Procurement & Supply, Technical/Engineering, Executive, IT/Admin.
* 11 explicitly Phase 2.
```

Why the department list is a regulatory artefact

Aviation regulations in most jurisdictions require an operator to appoint named holders of specific accountability positions — a safety officer, a quality manager, a technical manager. AMS must record who holds each role and what authority approved them, because that is an operating licence requirement, not a user-interface preference. This is why the roles appear in `domain.js` alongside the cost categories rather than in frontend configuration.

Spreadsheets, PDF and Word generation

These were discussed as a requirement and assessed as feasible. All three are *generation* — AMS produces a file — which is fundamentally different from hosting a collaborative editor, and none of it requires the server layer.

Output	How it would be produced	Verdict
CSV export	Written by hand — no dependency, opens in any spreadsheet application	Start here
Excel (.xlsx)	The SheetJS library — real multi-sheet workbooks, frozen headers, number formats	Viable, one dependency
PDF forms	The pdfkit library — streams a document without a browser	Viable, one dependency
Word (.docx)	The docx library	Viable, but see the caveat

```
// Why this is a pure function, like everything in shared/src/
export function budgetVarianceCsv(rows) {
  // 1. Validate input as integer cents
  // 2. Compute variance per row
  // 3. Emit CSV with no float formatting
  // 4. Test: assert the file round-trips and the columns sum to the total
}
```

The caveat on Word documents, and why it is not about technology

An editable .docx that people can modify is a document *outside* the audit chain. In a system holding records that regulators may require years later, "it was in the Word file" is not a defensible answer to "what did the approved maintenance manual say on this date?" If the document must not change, generate PDF and skip Word. If it must be editable, it belongs inside AMS with version history and audit trail.

```
// The safest dependency position: CSV by hand costs nothing.
const ESCAPE = (v) => `"${String(v).replace(/"/g, '""')}"`;
// Double the quote character inside a quoted field – RFC 4180.
export function toCsv(headers, rows) {
  return [
    headers.join(','),
    ...rows.map((r) => r.map(ESCAPE).join(',')),
  ].join('\r\n');
}
```

What still needs a qualified human

This is the most important section in the document, and the shortest, because it is where software ends.

Area	What software can do	What requires a qualified human
Cost taxonomy	Enforce the categories are valid and sum correctly	Decide whether the categories match how IFRS and IATA actually require reporting
Regulatory reserve	Enforce fuel never drops below the configured figure	Confirm the configured figure is legally correct for the jurisdiction
MEL intervals	Compute deadlines from the category table	Verify the category table matches current EASA CS-GEN-MMEL
AD applicability	Apply the serial range and threshold rules as given	Confirm the modelled applicability matches how a regulator actually determines it
Fuel burn factors	Apply the constants	Validate them against this fleet's measured data
Operating policy	Show the numbers	Decide what the airline should do about them

The boundary, stated precisely

Every test in AMS proves the code does what its authors intended. **No test can prove the intent was correct.** If the regulatory reserve requirement is wrong, all 190 tests pass and the aircraft is dispatched below its legal minimum. Software can enforce a rule; only a qualified person can establish that the rule is the right one.

```
// What the system can prove:  
// "Cumulative fuel on this route never exceeds capacity minus the  
//   configured reserve of 1500 kg."  
//  
// What it cannot prove:  
// "1500 kg is the legally correct reserve for this operation."
```

This is not a limitation to be engineered away. It is a permanent division of responsibility. The value of a system like AMS is that it makes the *computation* exact and the *decision* supportable — which means it makes the qualified human's judgement more valuable, not less.

Glossary

Terms used in this guide, in the order they appear. Where a term has both a general and an AMS-specific meaning, the AMS usage is given.

ASK (Available Seat Kilometres)

Seats offered multiplied by distance flown. The supply side of capacity, unaffected by ticket sales.

AD (Airworthiness Directive)

A mandatory instruction issued by a civil aviation regulator. Distinct from a Service Bulletin, which is not legally binding until adopted by an AD.

ATA chapter

The industry-standard equipment taxonomy — chapter 32 is landing gear, chapter 55 is powerplant. Engineers search compliance items by chapter, not by directive number.

Block hours

Time from door-close to door-open. The measure of aircraft flying time, and therefore of utilisation.

CASK (Cost per Available Seat Kilometre)

Operating cost divided by ASK. The industry's primary efficiency measure. Computed in AMS in microcents.

CASM

CASK expressed per Available Seat Mile rather than kilometre. Provided for US comparability.

Cents

The integer unit AMS uses for all money. Never a float. 123456 represents US\$1,234.56.

CTK (Cargo Tonne Kilometre)

The freight analogue of RPK — cargo mass multiplied by distance.

CORSIA

The ICAO global market-based measure for international aviation emissions, with thresholds and compliance periods.

EU ETS

The European Union Emissions Trading System, which covers intra-EU aviation and certain departures to the UK and Switzerland.

ETOPS reserve

Additional fuel required on long-range routes where a diversion would take the aircraft beyond the coverage of a suitable airport.

ETOPS

Extended-range Twin-engine Operational Safety Standards.

Final reserve

The fuel an aircraft must still have on landing — typically 30 minutes at holding speed. A hard regulatory minimum.

Flat allocation

Giving every flight the same share of a fixed cost. Crude but defensible for genuinely uniform costs; usually wrong.

FOR UPDATE

A database instruction that locks a row until the current transaction ends. The mechanism that makes double-spending unrepresentable in AMS.

Idempotency key

A unique value sent with a request so that a retry does not perform the work twice.

Idempotent

An operation that produces the same result whether applied once or many times.

IFRS

International Financial Reporting Standards. The reporting framework airlines use, including IFRS 16 on leases.

JSDoc

Comments in a special format that describe types and parameters. In AMS these drive the type-check gate without a compilation step.

Largest-remainder allocation

The method for splitting a total so the parts sum exactly to it, giving leftover units to the largest fractional shares. See `allocate()` in `money.js`.

LLP (Life-Limited Part)

A component with a hard life limit in cycles, hours or calendar months. Exceeding it requires destruction of the part.

Load factor (PLF)

Revenue passenger kilometres divided by available seat kilometres. Stored in AMS as integer parts per million.

MEL (Minimum Equipment List)

The list of items an aircraft may fly with unserviceable, subject to rectification deadlines by Category A, B, C or D.

Microcents

One hundredth of a cent. Used for unit costs so that a long-haul CASK does not round to zero.

Ppm (parts per million)

A rate expressed as an integer. 850,000 ppm is 85%. Used throughout AMS for ratios, thresholds and margin.

RASK

Revenue per available seat kilometre. The revenue side of the CASK comparison; the gap between them is the margin.

RPK (Revenue Passenger Kilometre)

Paying passengers multiplied by distance flown. The demand side, and the only passenger capacity measure that carries revenue.

SB (Service Bulletin)

A manufacturer instruction. Often technically equivalent to an AD but not legally mandatory on its own.

Safe integer

A number JavaScript can hold exactly. Up to 9,007,199,254,740,991. `Number.isSafeInteger` checks it.

Segment

One flight between two consecutive points.

Segregation of duties

The control requiring different people to authorise and approve the same transaction. See `SOD_RULES` in `domain.js`.

SHA-256

A hashing algorithm. Produces a 64-character fingerprint that changes completely if the input changes by one character.

TOCTOU

Time-of-check to time-of-use. The race where a value is checked, then changed before it is used. The reason `FOR UPDATE` is necessary.

Transaction

A set of database operations that either all succeed or all fail. AMS requires the commitment and its audit entry in one transaction.

ULL

Ultra Long Haul. Flights beyond roughly six hours.

Utilisation

Block hours flown per aircraft per day. The strongest single driver of unit cost.

Where to go from here

- **For a new engineer:** read Parts II and III, then the module headers in `shared/src/` in the order they appear in `costing.js`. Those headers are unusually good documentation and they explain intent.
- **For a reviewer:** read Part XIII, then open `shared/src/routing/constrained-path.js` and check the code against the proof in its header. That is the module most worth reviewing carefully.
- **For a manager:** you now know what the system measures, why the money is stored as integers, what the tests actually prove, and where a qualified human is still required. Ask for the open-questions list for your aviation finance and compliance reviewers before the system informs a real decision.